22/01/25

Types of Number systems

Name of NS	Base (or) Radix	Possible combination
1) Binary	2	0 & 1
2) Octal	8	0, 1, 2, 3, 4, 5, 6, & 7
3) Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8 & 9.
4) Hexa-Decimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9 A, B, C, D, E & F

Digital Data Representation

Decimal Digit	Binary (16 8 4 2 1)	Octal	Hexa-Decimal
0	00000	0	0
1	00001	1	1
2	00010	2	2
3	00011	3	3
4	00100	4	4
5	00101	5	5
6	00110	6	6
7	00111	7	7
8	01000	10	8
9	01001	11	9
10	01010	12	A
11	01011	13	B
12	01100	14	C
13	01101	15	D
14	01110	16	E
15	01111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12
19	10011	23	13
20	10100	24	14

Boolean Algebra Principles

Complement Law:- $\bar{A} = A$

Double-Negation Law:- $\bar{\bar{A}} = A$

Single Variable Theorems:- Complement & complement considered as one variable.

(a) $A + 0 = 0 + A = A$

(b) $A \cdot 1 = 1 \cdot A = A$

(c) $A + \bar{A} = 1$

(d) $A \cdot \bar{A} = 0$

(e) $A + A = A$

(f) $A + 1 = 1$

(g) $A \cdot 0 = 0$

Multi variable Theorems:-

(a) $A + B = B + A$

(b) $A \cdot B = B \cdot A$

(c) $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$

(d) $A + (B \cdot C) = (A + B) \cdot (A + C)$

(e) $A + A \cdot B = A(1 + B)$
$= A \cdot 1$
$= A$

(f) $\bar{A} + (A \cdot B) = (\bar{A} + A) \cdot (\bar{A} + B)$
$= 1 \cdot (\bar{A} + B)$
$= (\bar{A} + B)$

De-Morgan's theorem

① $\overline{A \cdot B} = \bar{A} + \bar{B}$

A	B	$A \cdot B$	$\overline{A \cdot B}$	$\bar{A}$	$\bar{B}$	$\bar{A} + \bar{B}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

② $\overline{A+B} = \bar{A} \cdot \bar{B}$

A	B	$A + B$	$\overline{A+B}$	$\bar{A}$	$\bar{B}$	$\bar{A} \cdot \bar{B}$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

25 - 11001

23/01/25

Boolean Properties

Name of the property	AND	OR
Commutative property	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative Property	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A+B)+C = A+(B+C)$
Distributive Property	$A \cdot (B+C) = (A \cdot B)+(A \cdot C)$	$A+(B \cdot C) = (A+B) \cdot (A+C)$
Identity Property	$A \cdot 1 = A$	$A + 0 = A$
Complement Property	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
De-Morgan's	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A+B} = \bar{A} \cdot \bar{B}$

Ex 1:- Find the complement of $f(x, y, z) = (x \cdot y) + (\bar{x}\, z) + (y \bar{z})$

$$\bar{F} = \overline{(x \cdot y) + (\bar{x} \cdot z) + (y \bar{z})}$$

$$= \overline{(x \cdot y)} \cdot \overline{(\bar{x} z)} \cdot \overline{(y \bar{z})}$$

$$= (\bar{x} + \bar{y}) \cdot (\bar{\bar{x}} + \bar{z}) \cdot (\bar{y} + \bar{\bar{z}})$$

$$= (\bar{x} + \bar{y}) \cdot (x + \bar{z}) \cdot (\bar{y} + z)$$

Ex 2:- find the complement of $f(A, B, C) = (\bar{A} + \bar{B}) \cdot (\bar{C} + \bar{D}) \cdot (E + F)$ (D, E, F)

$$\bar{F} = \overline{(\bar{A} + \bar{B}) \cdot (\bar{C} + \bar{D}) \cdot (E + F)}$$

$$= \overline{(\bar{A} + \bar{B})} + \overline{(\bar{C} + \bar{D})} + \overline{(\bar{E} + F)}$$

$$= \bar{\bar{A}} \cdot \bar{\bar{B}} + \bar{\bar{C}} \cdot \bar{\bar{D}} + \bar{E} \cdot \bar{F}$$

$$= A \cdot B + C \cdot D + \bar{E} \cdot \bar{F}$$

Ex 3:- Simplify the expression $(A+B) \cdot (A+C)$

Sol:- $A \cdot A + A \cdot C + A B + B C$

$A + AC + AB + BC$

$A(1 + C + B) + BC$

$A(1) + BC$

$A \cdot 1 + BC$

$A + B \cdot C$

Ex4:- Simplify the given Boolean expression

$A\bar{B}D + A\bar{B}\bar{D}$

Sol:- $= A\bar{B}(D + \bar{D})$

$= A\bar{B}(1) = A\bar{B}$

Ex5:- Simplify the given Boolean Expression

$Y = AB + A(B+C) + B(B+C)$

$Y = AB + (A \cdot B) + (A \cdot C) + (B \cdot B) + (B \cdot C)$

$Y = A(B + B + C) + (B \cdot B) + (B \cdot C)$

$Y = A(B + B + C) + B(B + C)$

$= A(B+C) + B(B+C)$

$= AB + AC + B + BC$

$= B(1 + A + C) + AC$

$= B + AC$

Ex6:- Simplify $Z = \bar{A}\bar{B}C + \overline{(A+B+C)} + \bar{A}\bar{B}\bar{C}D$.

$+\bar{A}\bar{B}C + \bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A}\bar{B}\bar{C}D$

$\bar{A} \cdot \bar{B}(C + \bar{C} + \bar{C}D)$

$\bar{A} \cdot \bar{B}(1 + \bar{C}D)$

$\bar{A} \cdot \bar{B}$.

SOP/POS representation

(A) Minterm

A minterm is a standard product which consist of all variables in either complemented or uncomplemented form for which the output is 1.

Ex:- $F(A,B,C) = \bar{A}\,\bar{B}\,C + A\,B\,\bar{C} + A\,B\,C + \bar{A}\,B\,C$

* $\bar{A} = 0$ } only in minterms representation.

(B) Maxterm

A maxterm is a standard sum which consist of all variables in either complemented or uncomplemented form for which the output is 1.

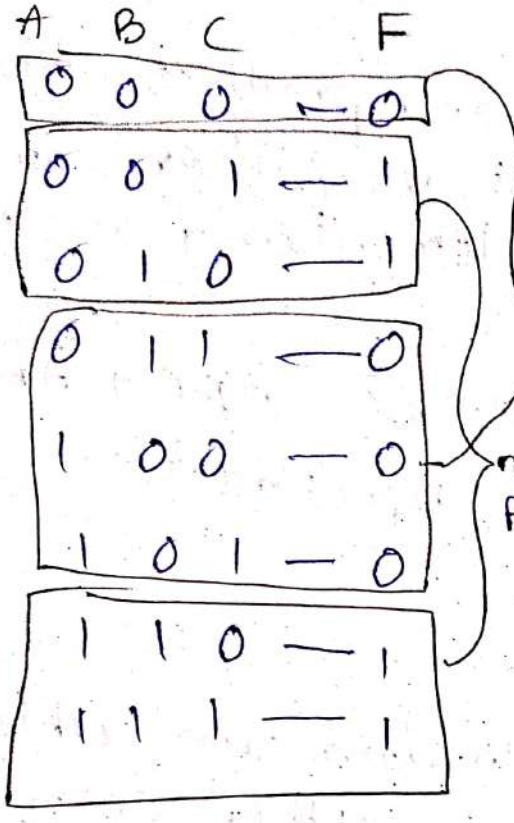
Ex:- $F(A,B,C) = (A+B+C)\cdot(\bar{A}+\bar{B}+C)\cdot(A+\bar{B}+\bar{C})$

* $\bar{A} = 0$ } in maxterm representation.

Minterms & Maxterms

Number	A	B	C	(SOP) Minterm	(POS) Maxterm
0	0	0	0	$\bar{A}\,\bar{B}\,\bar{C} = m_0$	$A+B+C = M_0$
1	0	0	1	$\bar{A}\,\bar{B}\,C = m_1$	$A+B+\bar{C} = M_1$
2	0	1	0	$\bar{A}\,B\,\bar{C} = m_2$	$A+\bar{B}+C = M_2$
3	0	1	1	$\bar{A}\,B\,C = m_3$	$A+\bar{B}+\bar{C} = M_3$
4	1	0	0	$A\,\bar{B}\,\bar{C} = m_4$	$\bar{A}+B+C = M_4$
5	1	0	1	$A\,\bar{B}\,C = m_5$	$\bar{A}+B+\bar{C} = M_5$
6	1	1	0	$A\,B\,\bar{C} = m_6$	$\bar{A}+\bar{B}+C = M_6$
7	1	1	1	$A\,B\,C = m_7$	$\bar{A}+\bar{B}+\bar{C} = M_7$

Representation of minterms & maxterms



maxterms

$f(A,B,C) = (A+B+C)\cdot(A+\bar{B}+C)\cdot(\bar{A}+B+C)\cdot(\bar{A}+B+\bar{C})$

$= M_0 \cdot M_3 \cdot M_4 \cdot M_5$

$\Pi M(0,3,4,5)$

minterms

$F(A,B,C) = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C}$

$= m_1 + m_2 + m_6 + m_7$

$= \Sigma m(1,2,6,7)$

Boolean Expressions

① $(A+B)\cdot(A+\bar{B})$

$(A\cdot A) + (A\cdot\bar{B}) + (A\cdot B) + (B\cdot\bar{B})$

$(A\cdot A) + (A\cdot\bar{B}) + (A\cdot B) + 0$

$A(A+\bar{B}+B)$

$A(A+1)$

$A(1)$

$= A$

② $A\cdot(B+C) + \bar{A}\cdot(B+C)$

$= A\bar{A}(B+C)$

$= 1(B+C)$

$= B+C$

(3) $(A+B)\cdot(A+C)\cdot(B+C)$

$A(B+C)\cdot(B+C)$

$\cdot(B+C)(A+1)$

$= A+(B+C)$

(4) $A\cdot B + A\cdot\bar{C} + B\cdot C$

$= A(B+\bar{C}) + (B\cdot C)$

$= A\cdot(B+\bar{C})$

(5) ~~$A\cdot\bar{B} + A\cdot B$~~

(5) $(A+B)\cdot(A+\bar{B}\cdot C)$

$= A(A+\bar{B}\cdot C) + B\cdot(A+\bar{B}\cdot C)$

$= AA + A\cdot\bar{B}\cdot C + B\cdot A + B\cdot\bar{B}\cdot C$

$= A + A\cdot\bar{B}\cdot C + B\cdot A + 0$

$= A + A(B+\bar{B}\cdot C)$

$= A + A(B+C)$

$= A$

Complement Expressions

(1) $\overline{A+B\cdot C}$

$= \bar{A}\cdot\overline{(B\cdot C)}$

$= \bar{A}\cdot(\bar{B}+\bar{C})$

(2) $\overline{A\cdot(B+C)}$

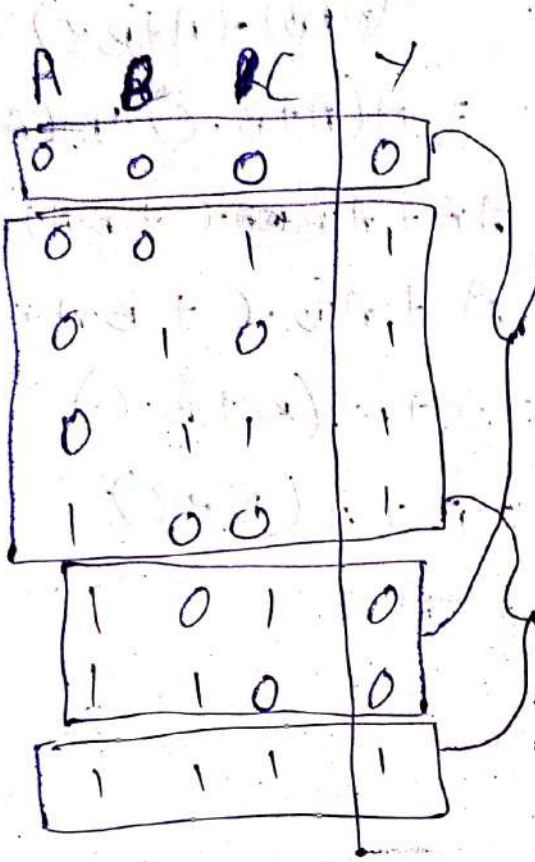
$= \bar{A} + \overline{(B+C)}$

$= \bar{A} + (\bar{B}\cdot\bar{C})$

(3) $\overline{\bar{A}+B}$

$= \bar{\bar{A}}\cdot\bar{B}$

$= A\cdot\bar{B}$



maxterms

$Y(A,B,C) = (A+B+C)\cdot(\bar{A}+B+\bar{C})\cdot(\bar{A}+\bar{B}+C)$

$= \prod m(0,5,6)$

minterms

$Y(A,B,C) = (\bar{A}\bar{B}C) + (\bar{A}B\bar{C}) + (\bar{A}BC) + (A\bar{B}\bar{C}) + (ABC)$

$= \Sigma m(1,2,3,4,7)$

Standard Form & Canonical Form

SOP → Canonical SOP form

Ex:- Represent the given expression in canonical SOP form.

$Y = AC + AB + BC$

$= AC(B+\bar{B}) + AB(C+\bar{C}) + BC(A+\bar{A})$

$= ABC + A\bar{B}C + ABC + AB\bar{C} + ABC + \bar{A}BC$

111 101 111 110 111 011

7 5 7 6 7 3

$= m_3 + m_5 + m_6 + m_7 = \Sigma m(3,5,6,7)$

Ex:- $F = \bar{A} + BC$

$BC \quad (A+\bar{A})$

$= \bar{A}(B+\bar{B})(C+\bar{C}) + BC(A+\bar{A})$

$= \cancel{\bar{A}BC + A\bar{B}C + \bar{A}BC}$

$(\bar{A}B + \bar{A}\bar{B})(C+\bar{C}) + ABC + \bar{A}BC$

$\bar{A}BC + \bar{A}B\bar{C} + \bar{A}\bar{B}C + \bar{A}\bar{B}\bar{C} + ABC + \bar{A}BC$

$011 \quad 010 \quad 001 \quad 000 \quad 111 \quad 011$

$3 \quad 2 \quad 1 \quad 0 \quad 7 \quad 3$

$= m_0 + m_1 + m_2 + m_3 + m_7 = \Sigma m(0,1,2,3,7)$

POS → Canonical POS form

Ex:- $F = (\bar{A}+B)\cdot(\bar{A}+C)$

$= (\bar{A}+B+C\cdot\bar{C})\cdot(\bar{A}+C+B\cdot\bar{B})$

$A \quad BC$

$A+BC$

By distributive law $(A+BC)$

$= (\bar{A}+B+C)\cdot(\bar{A}+B+\bar{C})\cdot(\bar{A}+B+C)\cdot(\bar{A}+\bar{B}+C)$

$100 \quad 101 \quad 100 \quad 110$

$4 \quad 5 \quad 4 \quad 6$

$= m_4 \cdot m_5 \cdot m_6$

$= \pi m(4,5,6)$

Ex2:- $F = (A+B)\cdot(B+C)\cdot(A+C)$

$= (A+B+C\cdot\bar{C})\cdot(B+C+A\cdot\bar{A})\cdot(A+C+B\cdot\bar{B})$

$0 \quad 1 \quad 0 \quad 7$

$= (\bar{A}+B+C)\cdot(\bar{A}+B+\bar{C})\cdot(A+B+C)\cdot(\bar{A}+B+C)\cdot$

$000 \quad 001 \quad 000 \quad 100$

$(A+B+C)\cdot(A+\bar{B}+C)$

$000 \quad 010$

$0 \quad 2$

$= \pi m(0,1,2,4)$

Karnaugh Map (K-map)

2-variable K-map structure

(A) SOP

A \ B	$\bar{B}$ (0)	B (1)
$\bar{A}$ (0)	$\bar{A}\cdot\bar{B}$ 0	$\bar{A}\cdot B$ 1
A (1)	$A\cdot\bar{B}$ 2	$A\cdot B$ 3

(B) POS

A \ B	$\bar{B}$ (0)	B (1)
A (0)	$A+B$ 0	$A+\bar{B}$ 1
$\bar{A}$ (1)	$\bar{A}+B$ 2	$\bar{A}+\bar{B}$ 3

3-variable K-map structure

(A) SOP

A \ BC	$\bar{B}\bar{C}$ (00)	$\bar{B}C$ (01)	BC (11)	$B\bar{C}$ (10)
$\bar{A}$ (0)	$\bar{A}\bar{B}\bar{C}$ 0	$\bar{A}\bar{B}C$ 1	$\bar{A}BC$ 3	$\bar{A}B\bar{C}$ 2
A (1)	$A\bar{B}\bar{C}$ 4	$A\bar{B}C$ 5	ABC 7	$AB\bar{C}$ 6

(B) POS

A \ BC	BC (00)	$B\bar{C}$ (01)	$\bar{B}\bar{C}$ (11)	$\bar{B}C$ (10)
A (0)	$A+B+C$ 0	$A+B+\bar{C}$ 1	$A+\bar{B}+\bar{C}$ 3	$A+\bar{B}+C$ 2
$\bar{A}$ (1)	$\bar{A}+B+C$ 4	$\bar{A}+B+\bar{C}$ 5	$\bar{A}+\bar{B}+\bar{C}$ 7	$\bar{A}+\bar{B}+C$ 6

4-variable K-map structure

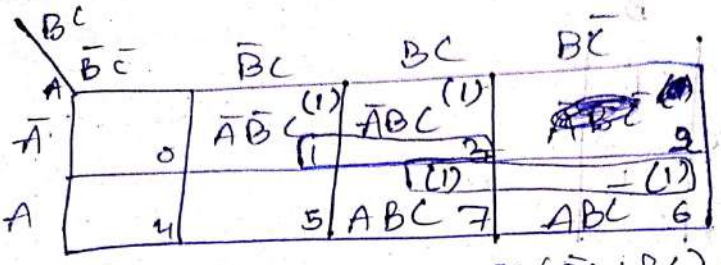
(A) SOP

AB \ CD	$\bar{C}\bar{D}$ (00)	$\bar{C}D$ (01)	CD (11)	$C\bar{D}$ (10)
$\bar{A}\bar{B}$ (00)	$\bar{A}\bar{B}\bar{C}\bar{D}$ 0	$\bar{A}\bar{B}\bar{C}D$ 1	$\bar{A}\bar{B}CD$ 3	$\bar{A}\bar{B}C\bar{D}$ 2
$\bar{A}B$ (01)	$\bar{A}B\bar{C}\bar{D}$ 4	$\bar{A}B\bar{C}D$ 5	$\bar{A}BCD$ 7	$\bar{A}BC\bar{D}$ 6
AB (11)	$AB\bar{C}\bar{D}$ 12	$AB\bar{C}D$ 13	$ABCD$ 15	$ABC\bar{D}$ 14
$A\bar{B}$ (10)	$A\bar{B}\bar{C}\bar{D}$ 8	$A\bar{B}\bar{C}D$ 9	$A\bar{B}CD$ 11	$A\bar{B}C\bar{D}$ 10

(B) POS

AB \ CD	CD (00)	$C\bar{D}$ (01)	$\bar{C}\bar{D}$ (11)	$\bar{C}D$ (10)
AB (00)	$A+B+C+D$ 0	$A+B+C+\bar{D}$ 1	$A+B+\bar{C}+\bar{D}$ 3	$A+B+\bar{C}+D$ 2
$A\bar{B}$ (01)	$A+\bar{B}+C+D$ 4	$A+\bar{B}+C+\bar{D}$ 5	$A+\bar{B}+\bar{C}+\bar{D}$ 7	$A+\bar{B}+\bar{C}+D$ 6
$\bar{A}\bar{B}$ (11)	$\bar{A}+\bar{B}+C+D$ 12	$\bar{A}+\bar{B}+C+\bar{D}$ 13	$\bar{A}+\bar{B}+\bar{C}+\bar{D}$ 15	$\bar{A}+\bar{B}+\bar{C}+D$ 14
$\bar{A}B$ (10)	$\bar{A}+B+C+D$ 8	$\bar{A}+B+C+\bar{D}$ 9	$\bar{A}+B+\bar{C}+\bar{D}$ 11	$\bar{A}+B+\bar{C}+D$ 10

Ex1:- Given $Z = \Sigma m(1,3,6,7)$ Minimize using 3-variable K-map.



Grouping:-

8 → ~~Quad~~ octent

4 → Quad

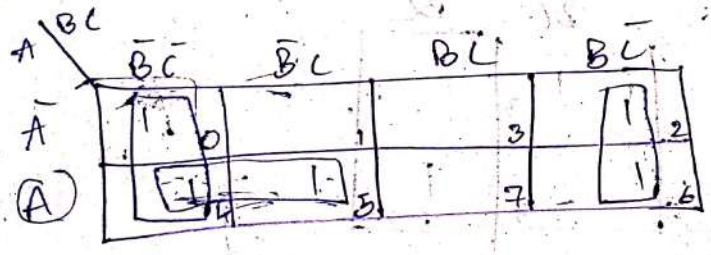
2 → Pair

$= \bar{A}(\bar{B}C + B\bar{C}) + A(BC + B\bar{C})$

$= \bar{A}C(B+\bar{B}) + AB(C+\bar{C})$

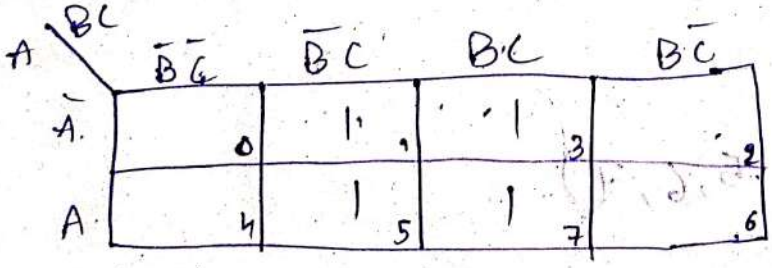
$= \bar{A}C + AB$

Ex2:- $F(A,B,C) = \Sigma m(0,2,4,5,6)$



$= A\bar{B} + \bar{C}$

Ex:- $F = \Sigma m(1,3,5,7)$



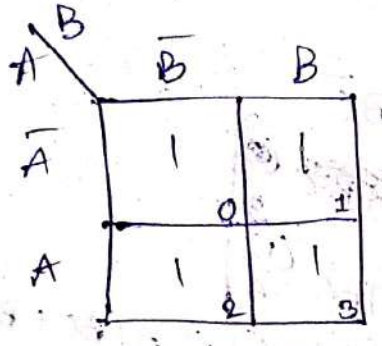
$\bar{A}(\bar{B}C + BC) + A(\bar{B}C + BC)$

$(\bar{B}C + BC)(\bar{A} + A)$

$(\bar{B}C + BC) \cdot 1$

$= C(\bar{B} + B) = C \cdot (1)$

$F = \Sigma m(0, 1, 2, 3)$

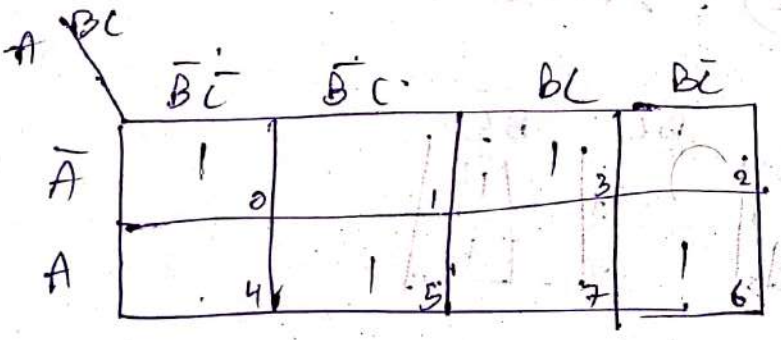


$\bar{A}(\bar{B}+B) + A(\bar{B}+B)$

$\bar{A}(1) + A(1)$

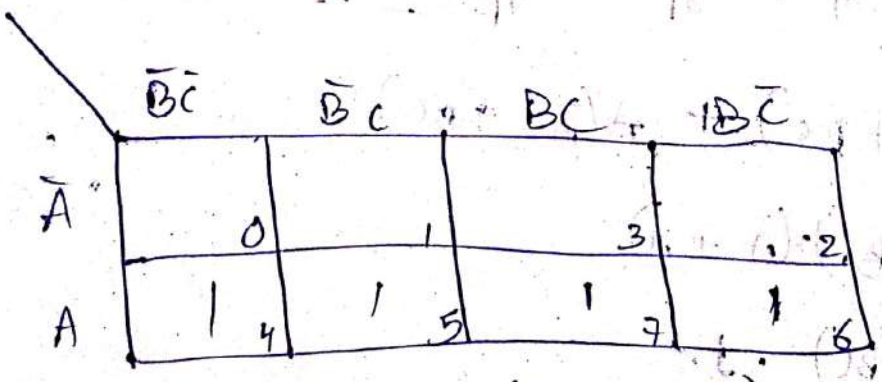
$\bar{A} + A = 1$

$F = \Sigma m(0, 3, 5, 6)$



$\bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}C + AB\bar{C}$

$F = \Sigma m(4, 5, 6, 7)$



$= A(\bar{B}\bar{C} + \bar{B}C) + A(BC + B\bar{C})$

$= A\bar{B}(1) + AB(1) \Rightarrow A\bar{B} + AB = A$

HALF ADDER

* It adds two bits of data and generate sum and carry.

Truth Table

A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

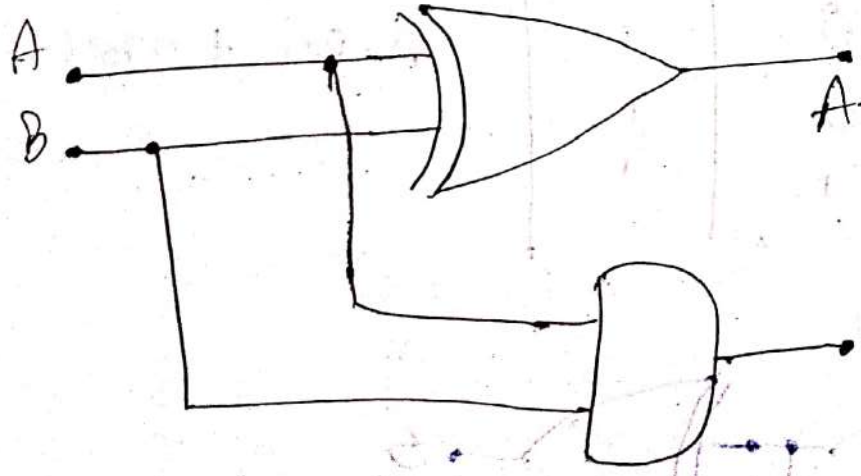
Expressions

Sum = $A \oplus B$

Carry = AB

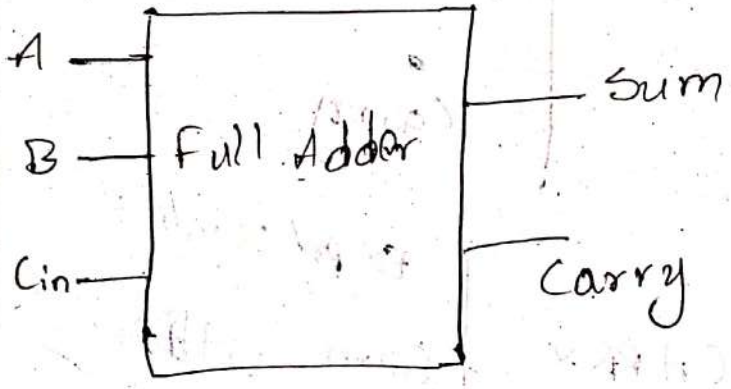
Sum = $\bar{A}B + A\bar{B}$

Logic Diagram



FULL ADDER

→ Full adder accepts two inputs bits a one input carry and generates a sum output & output carry.



* It adds three bits of data an generate sum of and carry.

Truth Table:-

A	B	C	SUM	CARRY
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Expressions

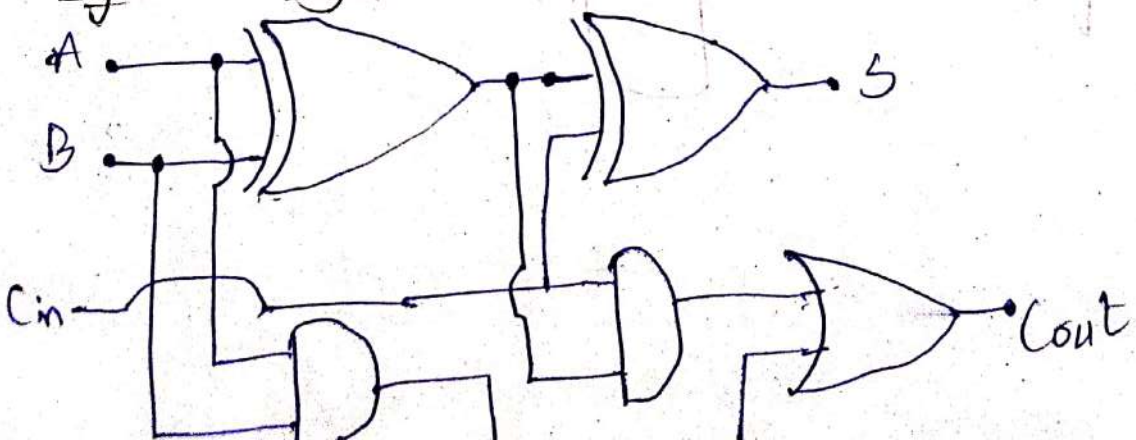
$Sum = \Sigma m(1,2,4,7$

$= \bar{A} \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot \bar{C} + A \cdot \bar{B} \cdot \bar{C} + A \cdot B \cdot C$

$Carry = \Sigma m(3,5,6,7$

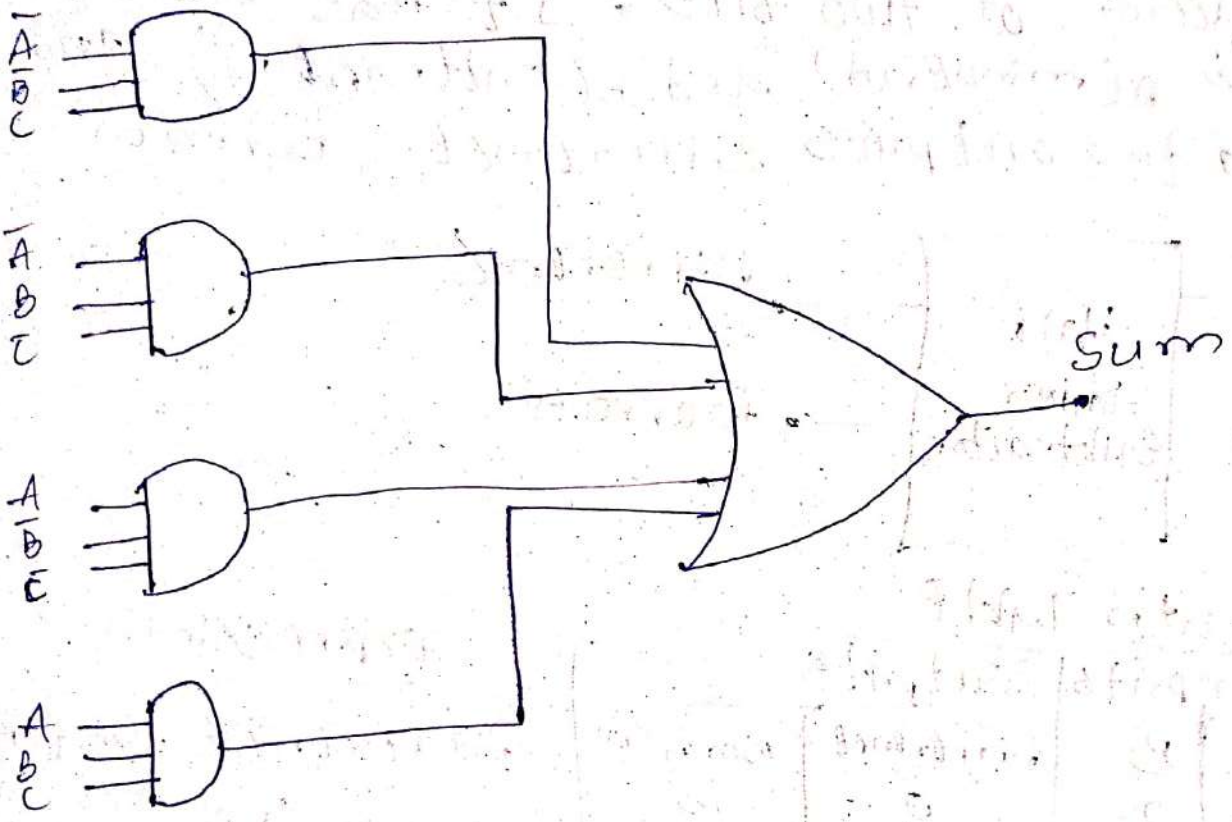
$= \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C} + A \cdot B \cdot C$

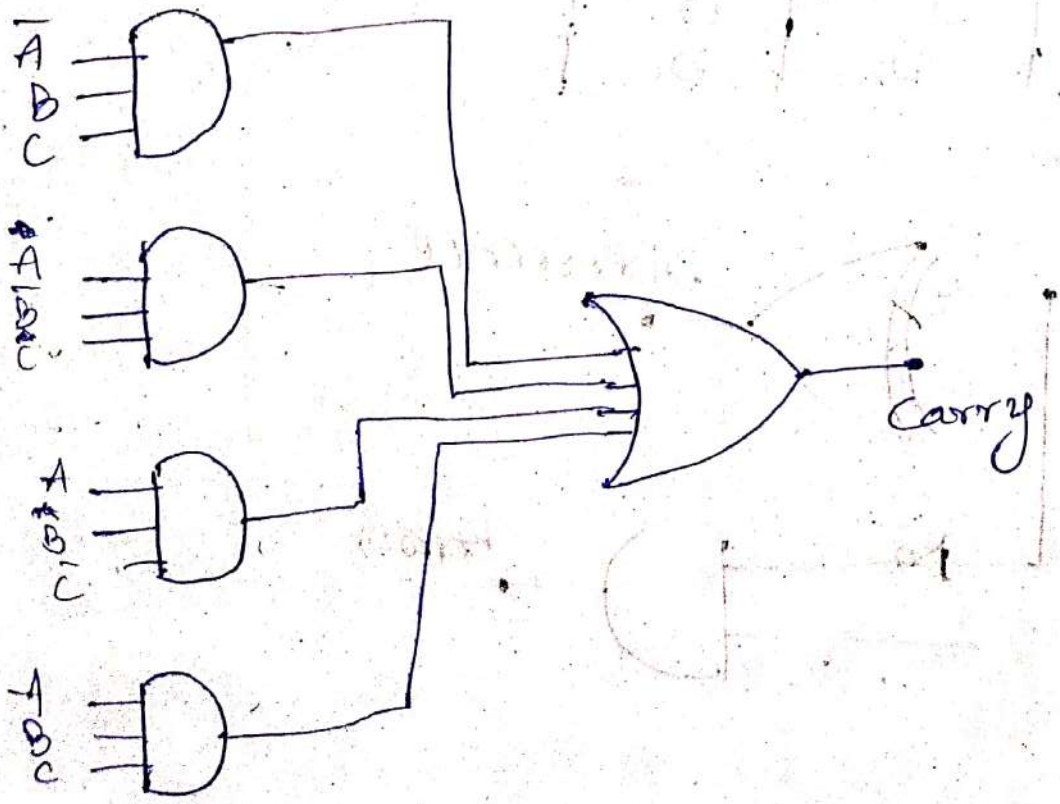
Logic Diagram



$Sum = \bar{A} \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot \bar{C} + A \cdot \bar{B} \cdot \bar{C} + A \cdot B \cdot C$

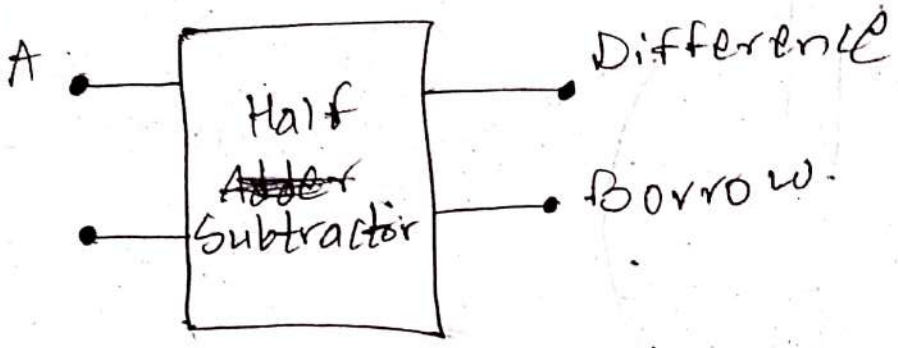
$Carry = \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C} + A \cdot B \cdot C$





Half subtractor

* the half subtractor is a combinational circuit which is used to perform subtraction of two bits. It has two inputs, A(minuend) and B(subtrahend) and two outputs difference & borrow.



Truth Table

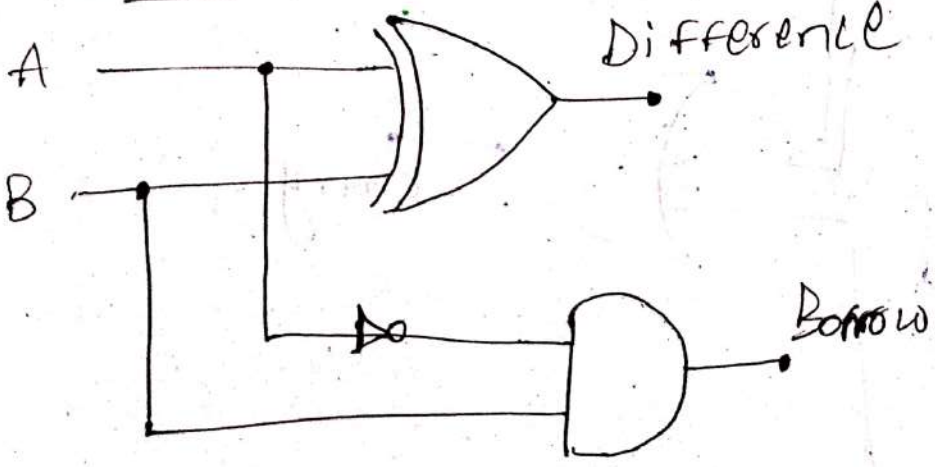
Inputs		Outputs	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Expressions

Difference = $A \oplus B$

Borrow = $\bar{A} \cdot B$

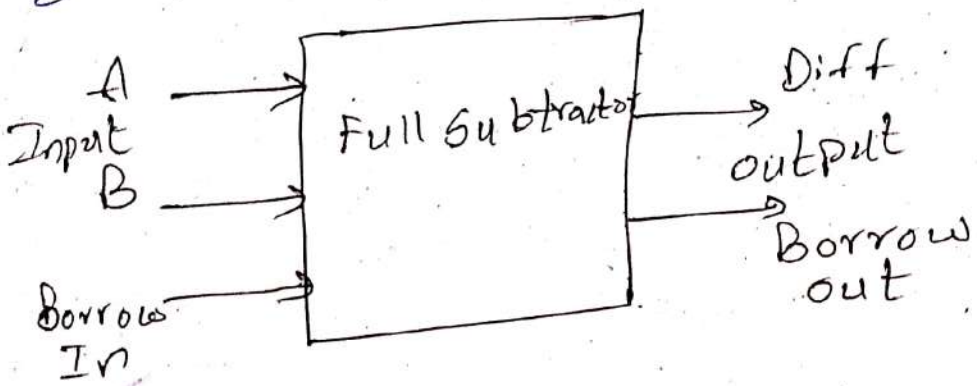
Diagram



FULL SUBTRACTOR

→ A full subtractor is a combinational circuit that performs subtraction involving three bits, namely A, B, Bin

* 3 inputs & 2 inputs.



Truth Table:-

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Expressions

$D = A \oplus B \oplus Bin$

$Bout = \bar{A}Bin + \bar{A}B + BBin$

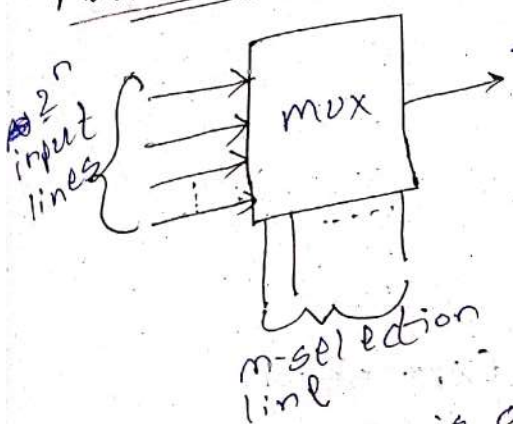
→ Output is '1' if the number of 1's appear odd times among the three inputs.

→ Output is '0' if the number of 1's appear even times among the three inputs.

For D (Difference)

→ For borrow Bout the output is 1 if $\bar{A}B$ (A=0 & B=1). Also if Bin=1 if the two inputs are '0'

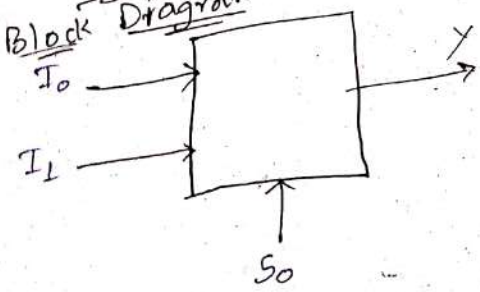
MULTIPLEXER



*A multiplexer is a combinational circuit that consist of 'm' selection lines, 2^m input lines and single output line.

2 X 1 multiplexer

Block Diagram



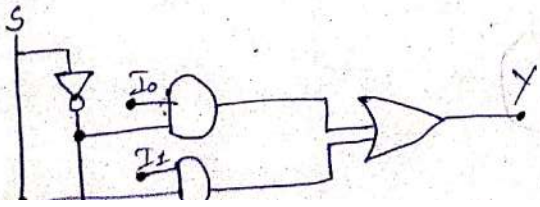
Truth Table

I_0	I_1	Z
0		

S	Y
0	I_0
1	I_1

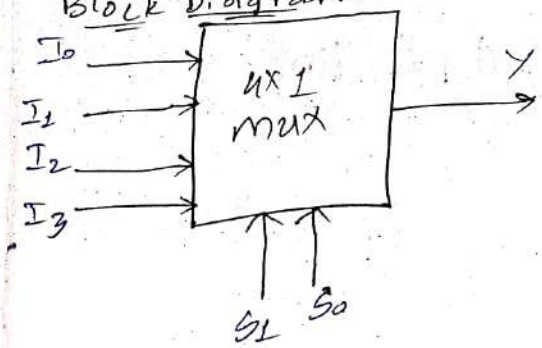
$Y = I_0 \cdot \bar{S} + I_1 S$

Circuit Diagram



4x1 multiplexer

Block Diagram



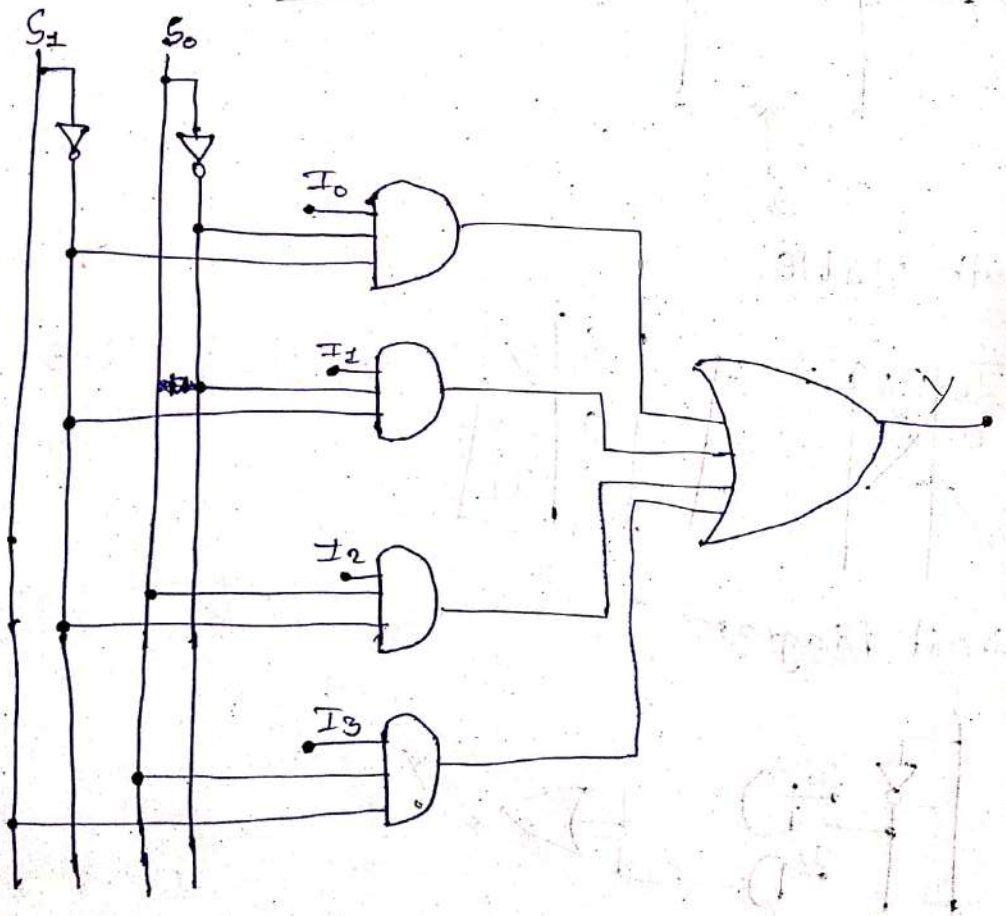
Truth Table

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Expression

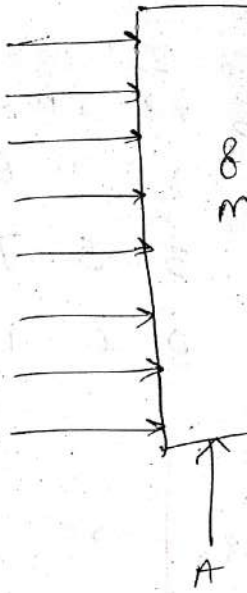
$Y = I_0 \bar{S_0} \bar{S_1} + I_1 S_0 \bar{S_1} + I_2 S_0 \bar{S_1} + I_3 S_0 S_1$

Circuit Diagram



Q. Implement t
using suitable

Sol:- Block Diag

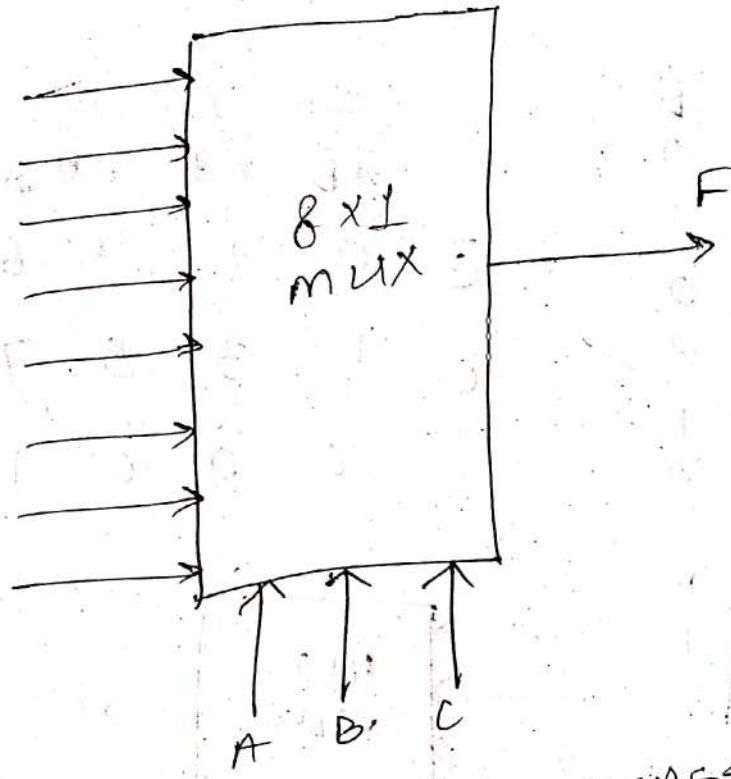


Truth Table

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	
1	1	1	

Q. Implement the $F(A,B,C) = \Sigma m(0,1,4,6,7)$ using suitable Mux.

Sol:- Block Diagram



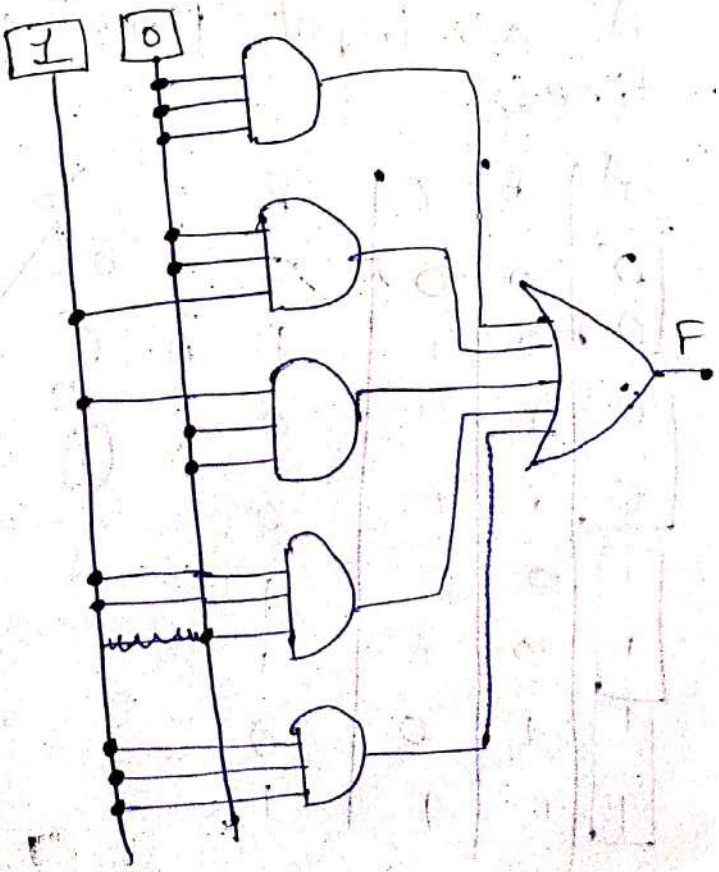
Truth Table

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Expression

$F = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}\bar{C} + AB\bar{C} + ABC$

Circuit Diagram



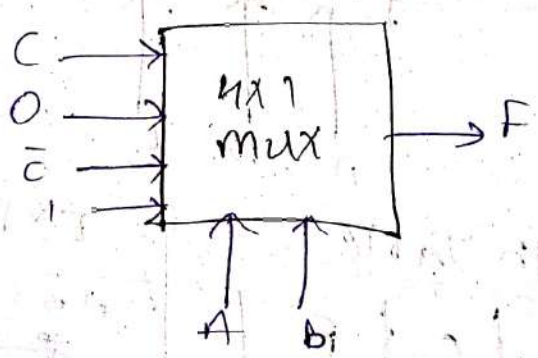
Q. Implement the function $F(A,B,C) = \Sigma m(1,4,6,7)$ using 4x1 mux considering 'C' as input line and A, B as selection lines.

Sol:-

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

	I_0 $\bar{A}\bar{B}$	I_1 $\bar{A}B$	I_2 $A\bar{B}$	I_3 AB
$\bar{C}$	0	2	4	6
C	1	3	5	7

$C \quad 0 \quad \bar{C} \quad 1$



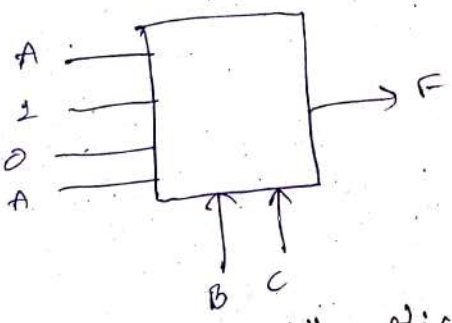
Q. Design the function $F(A,B,C) = \Sigma m(1,4,5,7)$ using 4x1 mux considering 'A' as input line & B, C as selection lines.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

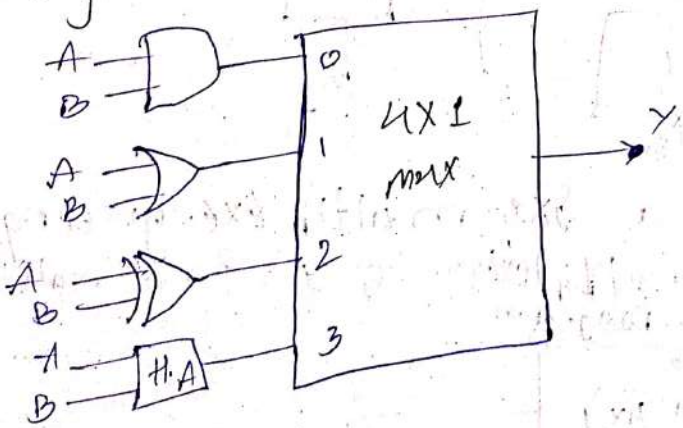
	I_0 $\bar{B}\bar{C}$	I_1 $\bar{B}C$	I_2 $B\bar{C}$	I_3 BC
$\bar{A}$	0	2	4	6
A	1	3	5	7

$A \quad 0 \quad 1 \quad A$

	I_0 $\bar{B}\bar{C}$	I_1 $\bar{B}C$	I_2 $B\bar{C}$	I_3 BC
$\bar{A}$	0	1	2	3
A	4	5	6	7
	A	1	0	A



Q. Develop an Arithmetic Logic Unit by using a 4x1 mux.



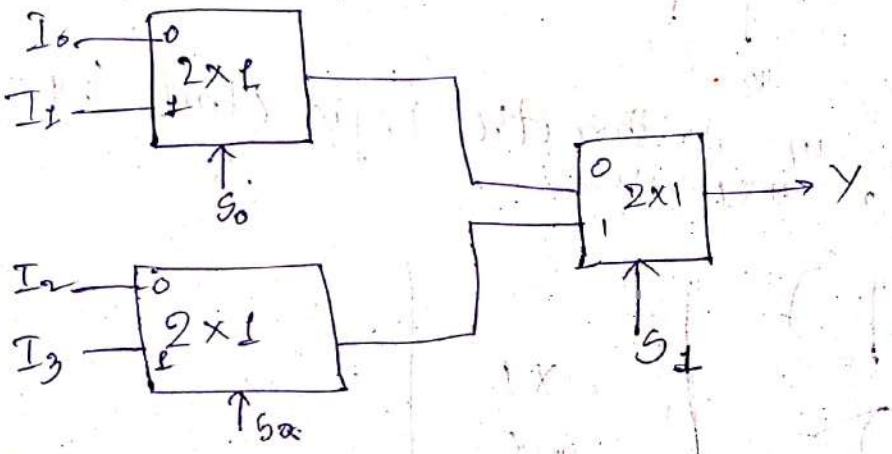
S_1	S_0	
0	0	AND
0	1	OR
1	0	XOR
1	1	Half Adder

Q Design a 4x1 multiplexer using 2x1 multiplexer.

Truth Table

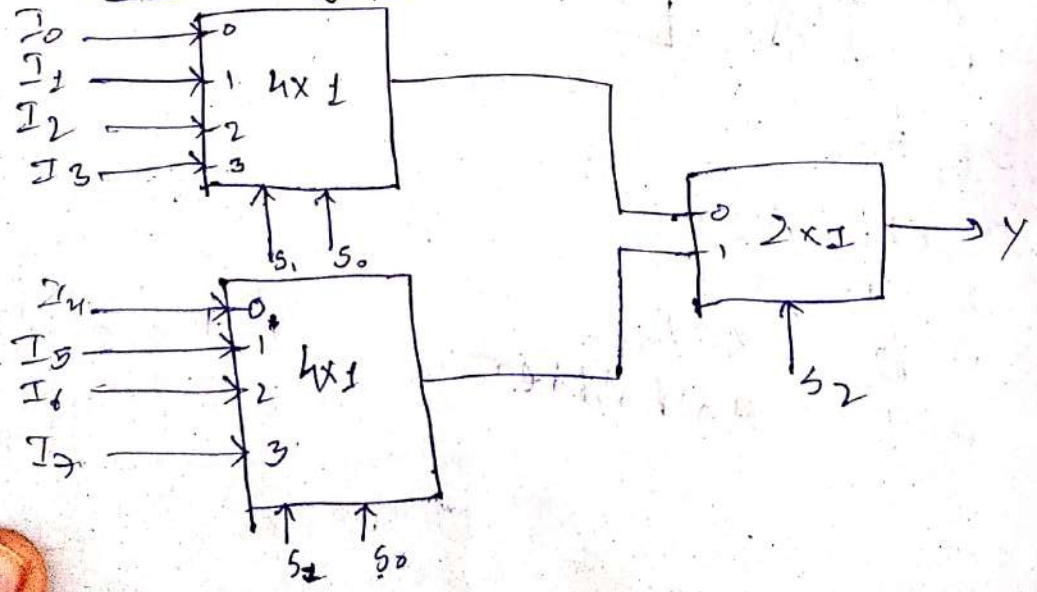
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Circuit Diagram:-



Q. Design a 8x1 multiplexer using two 4x1 multiplexers & 1 2x1 multiplexer.

Circuit Diagram



Truth Tabl

S_2	S_1	
0	0	
0	0	
0	1	
0	1	
1	0	
1	0	
1	1	
1	1	

Demultip

y input lines.

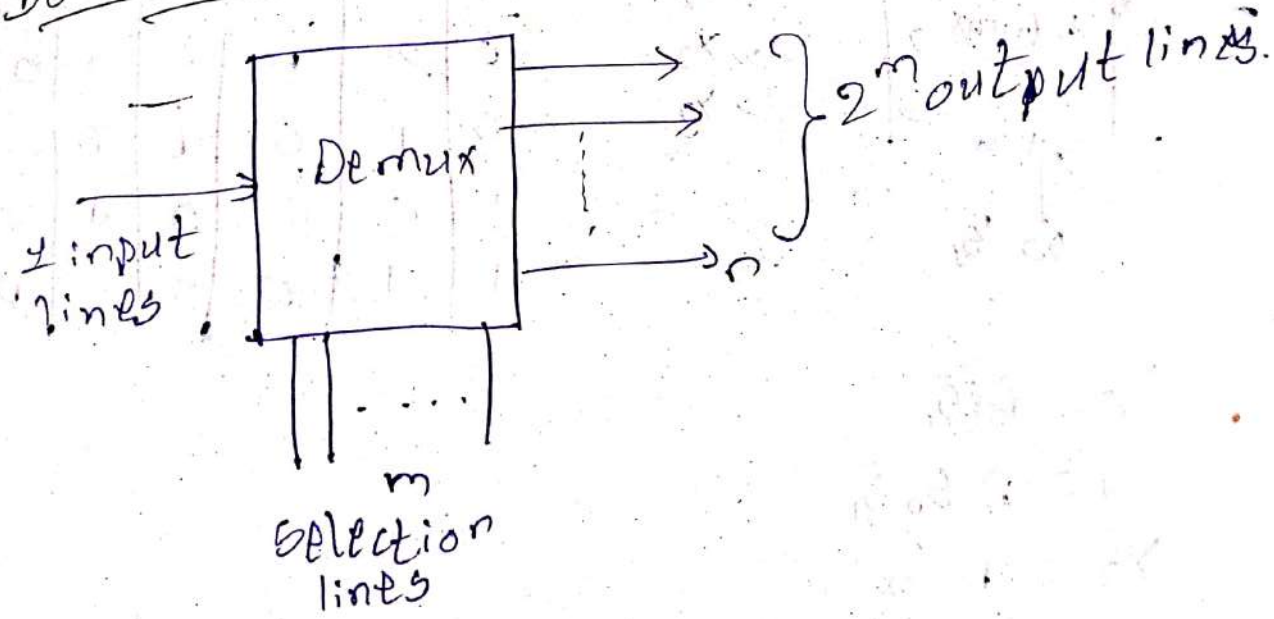
1x2 De

D

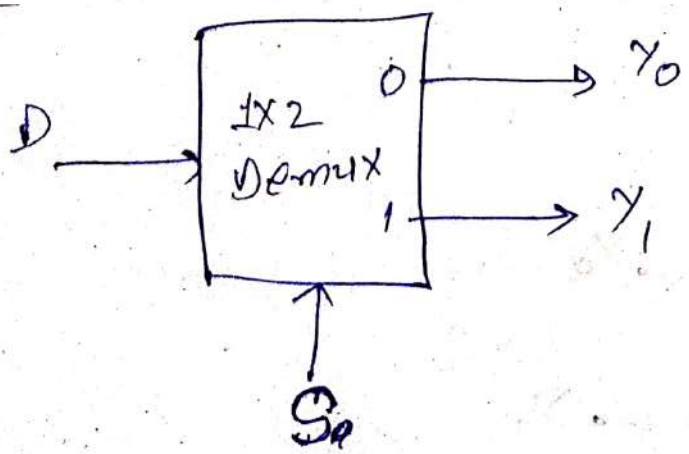
Truth Table

S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Demultiplexer



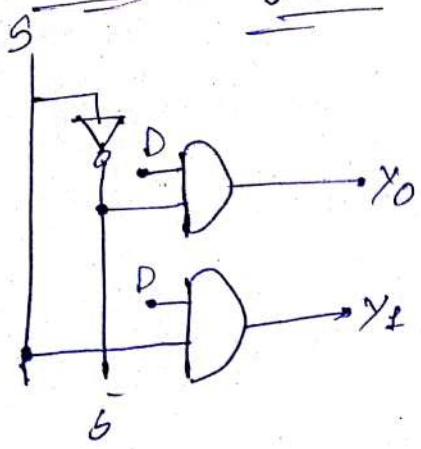
1x2 Demux



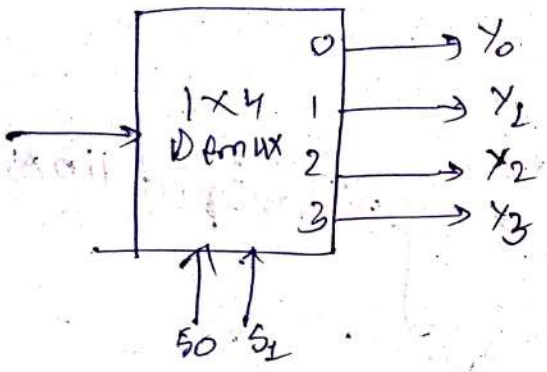
Truth Table

S	Y_0	Y_1
0	D	0
1	0	D

Circuit Diagram



1x4 Demux



Truth table

S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

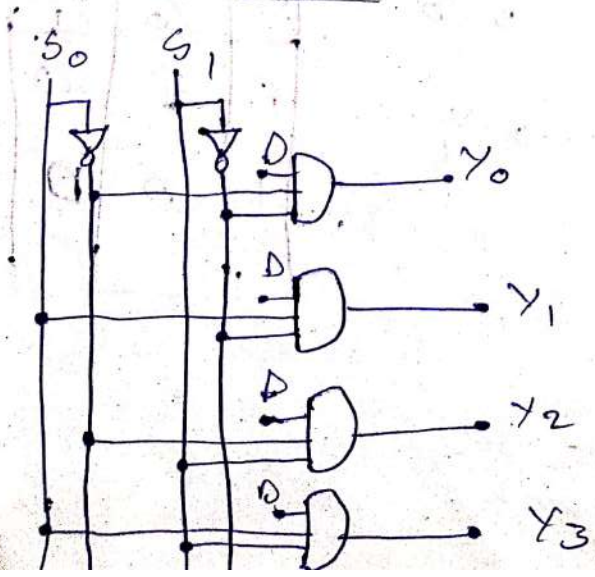
$Y_0 = D\bar{S}_0\bar{S}_1$

$Y_1 = D S_0 \bar{S}_1$

$Y_2 = D \bar{S}_0 S_1$

$Y_3 = D S_0 S_1$

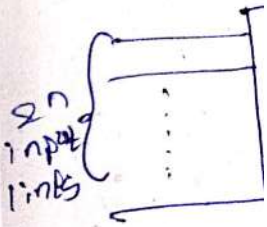
Circuit Diagram



Encoder

An enco
character
format.
lines an

Block Di



8 x 3 En

Bl

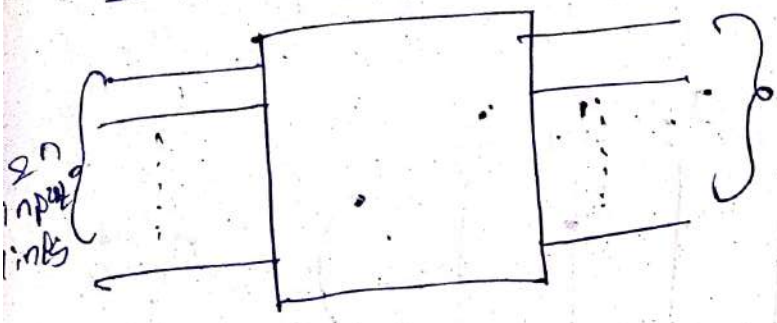
Truth

I_7	I_6
0	0
0	0
0	0
0	0
0	0
0	0
0	1
1	0

Encoder

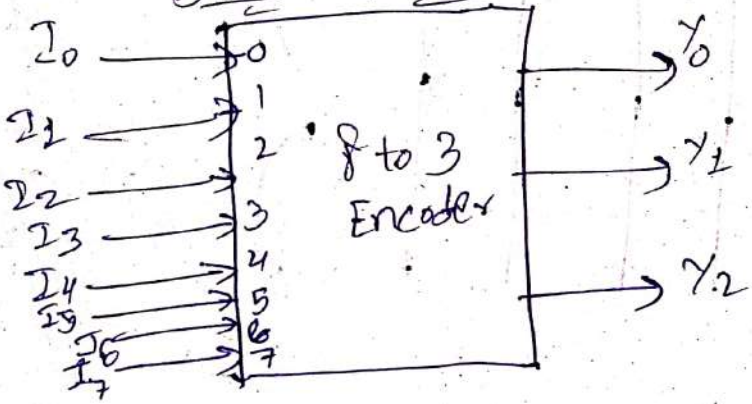
An encoder which converts numbers or characters or symbols into a coded format. It has a max of 2^n input lines and 'n' output lines.

Block Diagram



8 x 3 Encoder

Block Diagram



Applications:-

Used in Encoder Navigation.

Truth Table

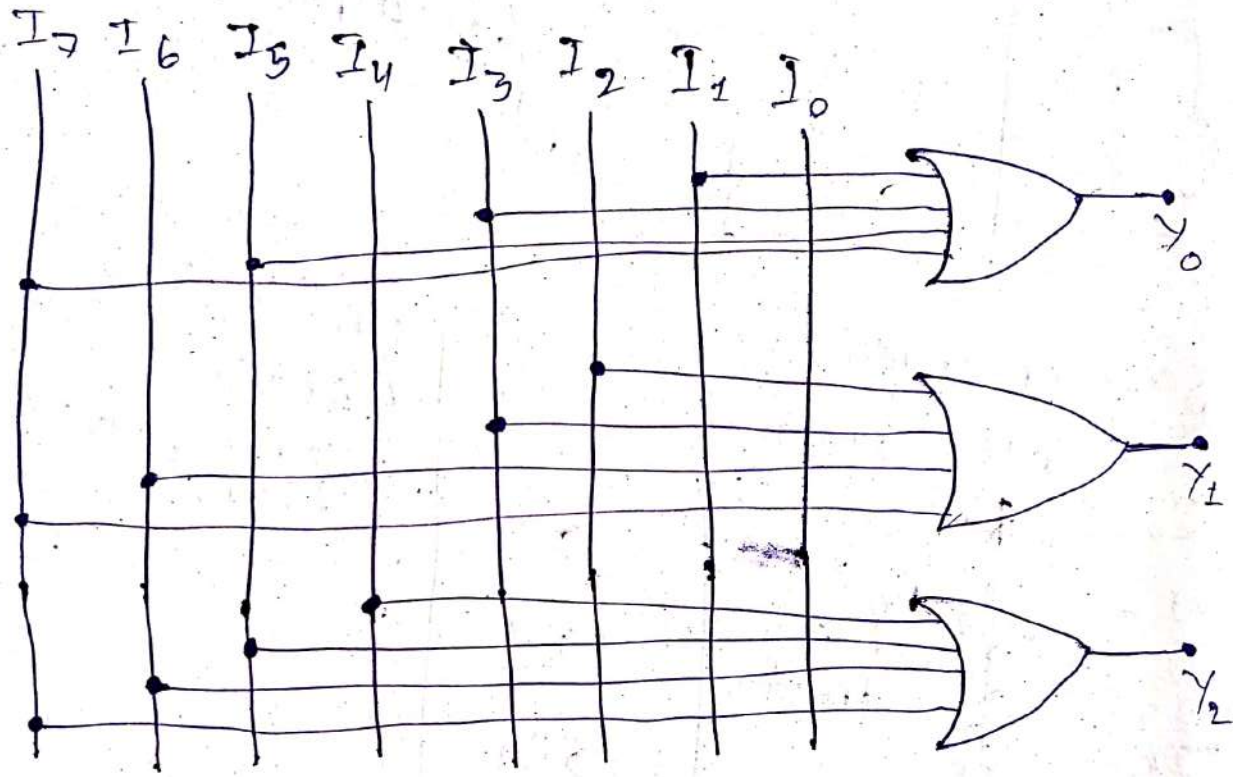
I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$Y_0 = \Sigma m(1,3,5,7) = I_1 + I_3 + I_5 + I_7$

$Y_1 = \Sigma m(2,3,6,7) = I_2 + I_3 + I_6 + I_7$

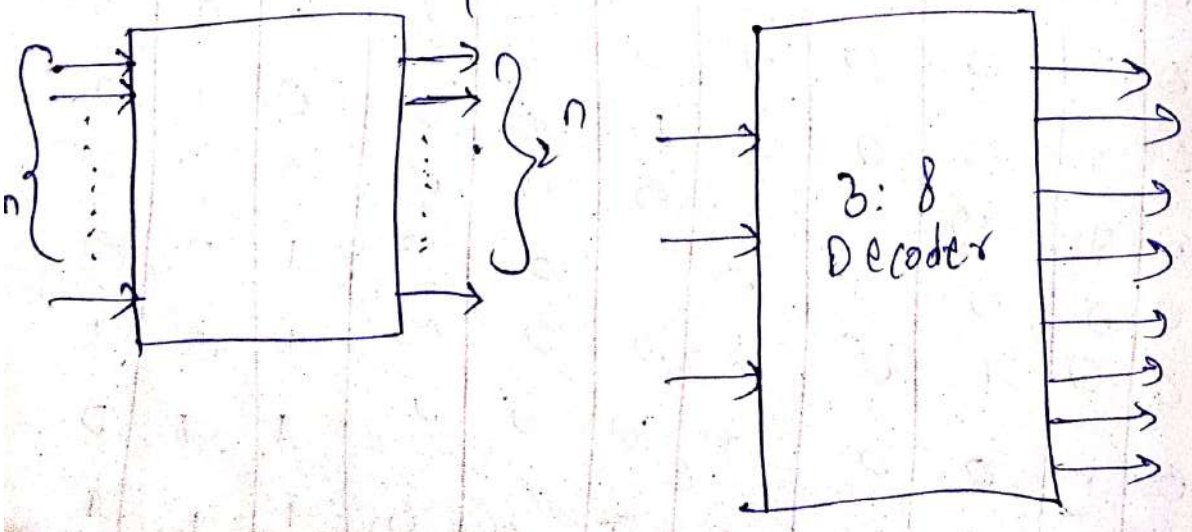
$Y_2 = \Sigma m(4,5,6,7) = I_4 + I_5 + I_6 + I_7$

Circuit Diagram



3 x 8 Decoder

Decoder is a combinational circuit that has 'n' input lines and max of 2^n output lines & 'n' input lines.



Truth Table

x	y	z	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

$Y_0 = \bar{x}\,\bar{y}\,\bar{z}$

$Y_1 = \bar{x}\,\bar{y}\,z$

$Y_2 = \bar{x}\,y\,\bar{z}$

$Y_3 = \bar{x}\,y\,z$

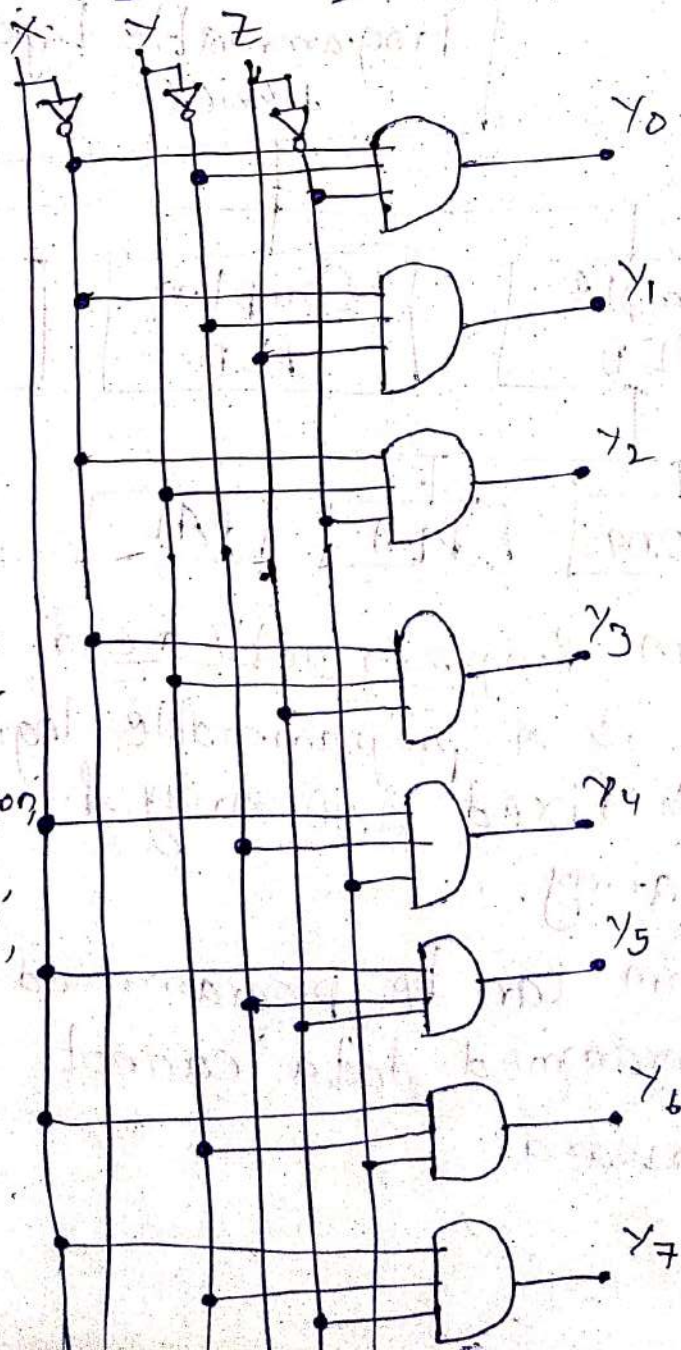
$Y_4 = x\,\bar{y}\,\bar{z}$

$Y_5 = x\,\bar{y}\,z$

$Y_6 = x\,y\,\bar{z}$

$Y_7 = x\,y\,z$

Circuit Diagram



Application:-

Addition, Subtraction, Multiplication, Division, AND, OR, NOT, XOR.

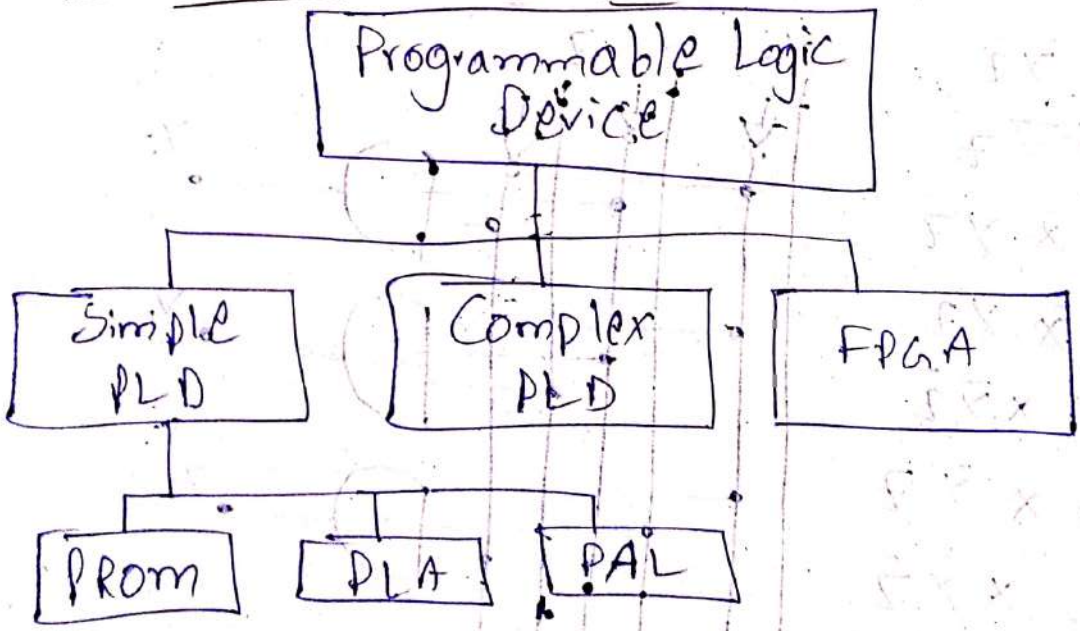
PLD Programmable Logic Devices

→ It is a type of digital integrated circuit that can be programmed or configured by the user to implement various digital logic functions.

Key characteristics of PLD's include:

* Programmability
* Reprogramonability
* Configure Logic Blocks (CLB's)
* Interconnectivity.

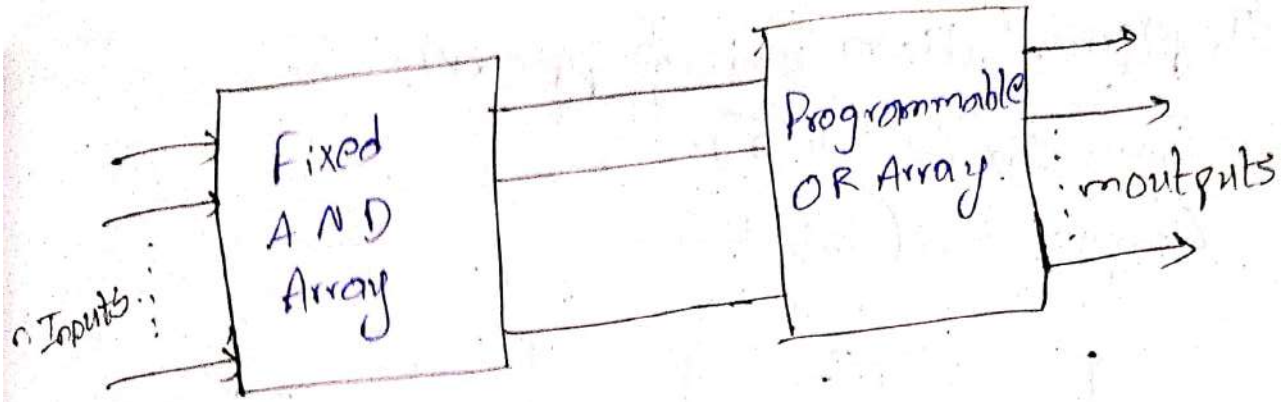
Classification of PLD's



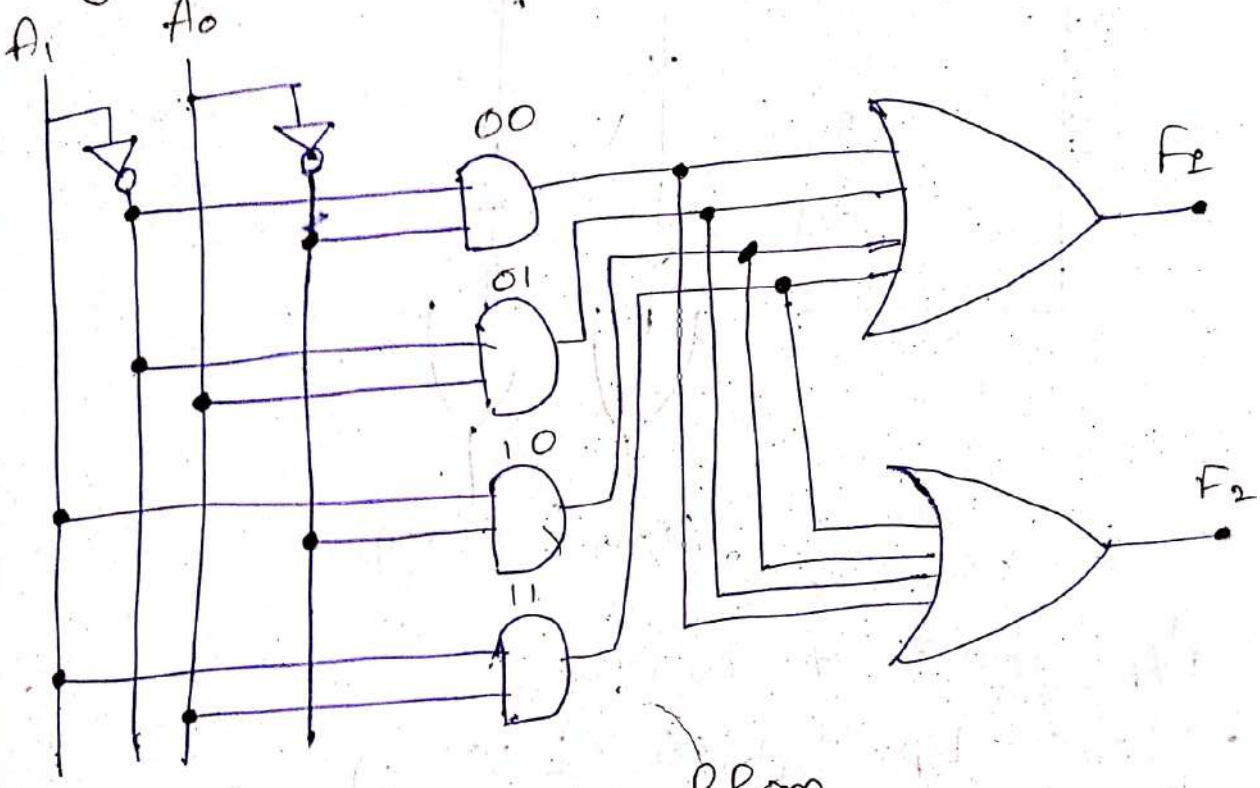
PROM Programmable Read only Memory

→ It is a programmable logic device that has fixed AND array & Programmable OR array.

→ PROM can be programmed once, and the programmed data cannot be changed afterward.

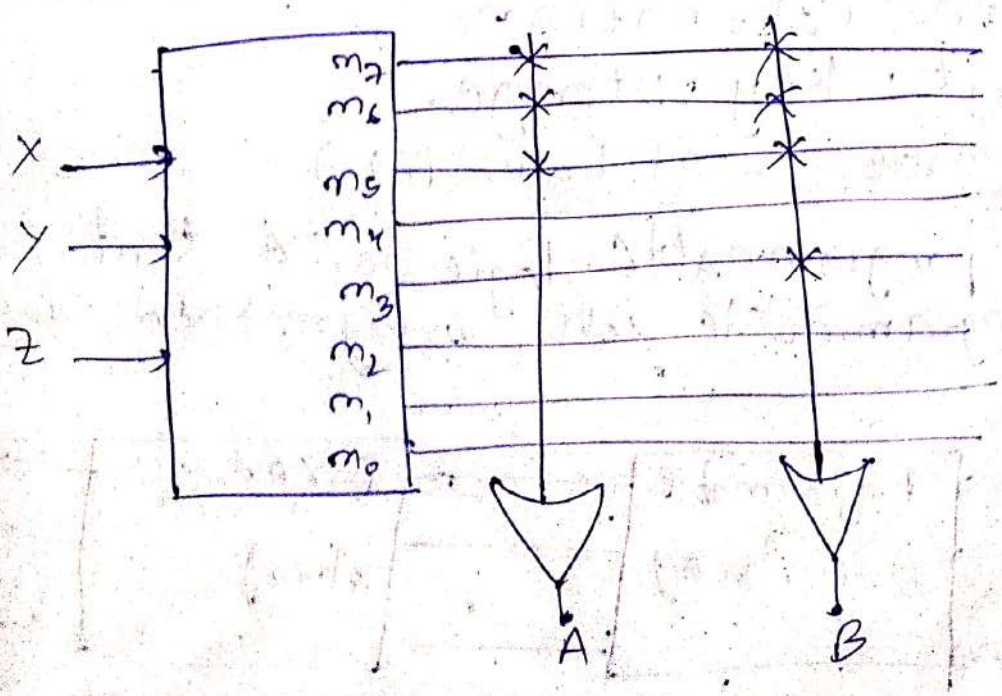


Logic Diagram



Implementation using PROM

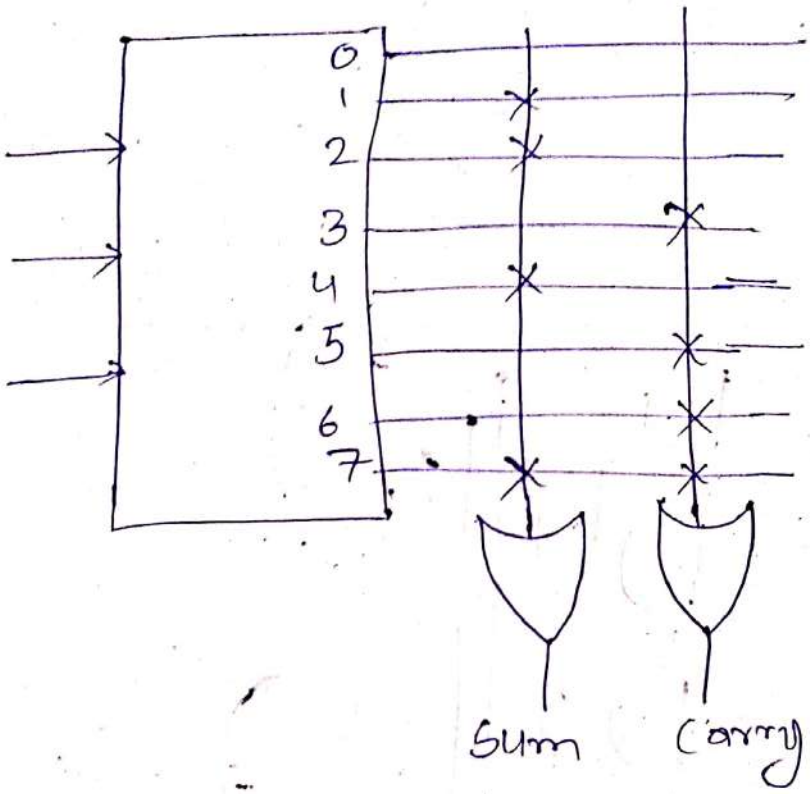
① $A(X,Y,Z) = \sum m(5,6,7)$ $B(X,Y,Z) = \sum m(3,5,6,7)$



Implementation using PROM

Sum = $\Sigma m(1, 2, 4, 7)$

Carry = $\Sigma m(3, 5, 6, 7)$

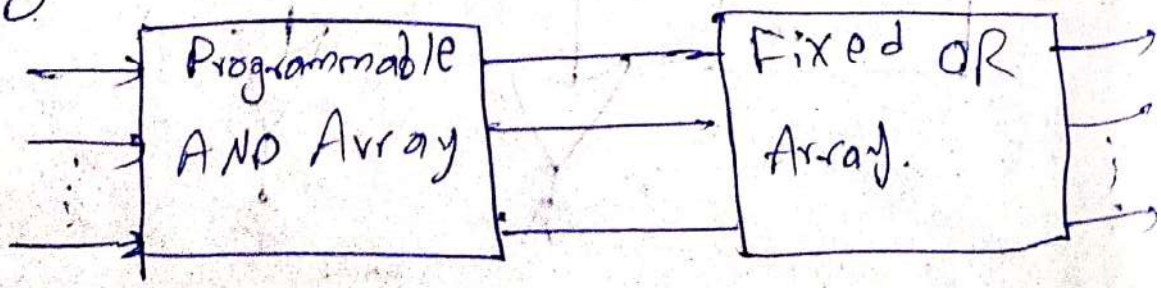


Applications of PROM

→ Look-Up-Tables (LUTs)
→ Custom Logic Functions
→ Mux & Demux Coders.
→ Digital Calibration.
→ Security key storage.

Programmable Array Logic (PAL)

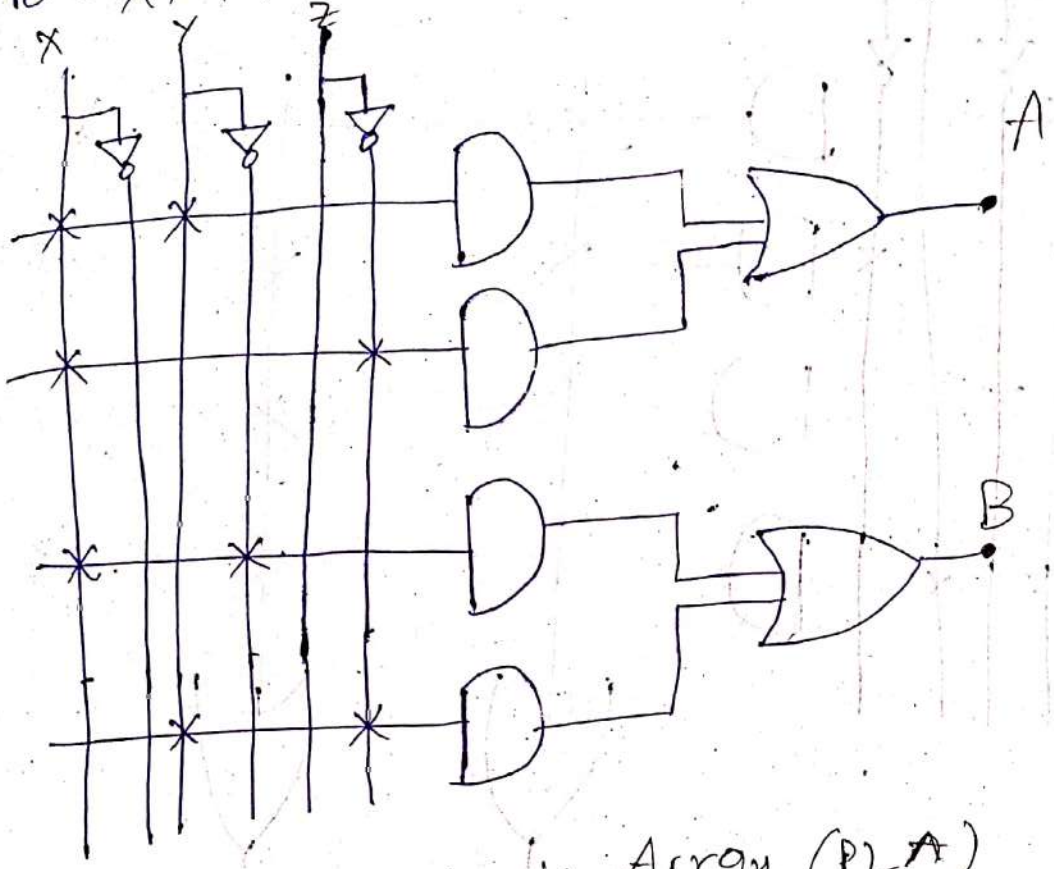
→ PAL is programmable logic device that has Programmable AND array & Fixed OR array.



Implementation using PAL

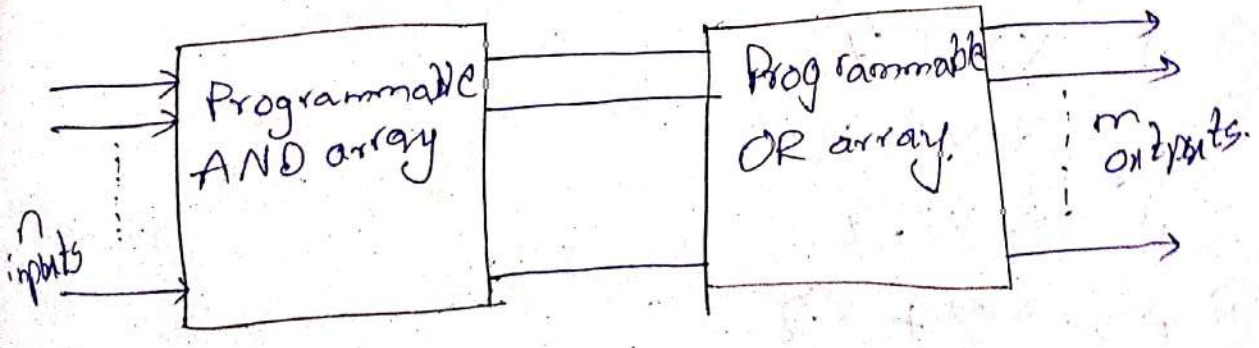
$A = XY + X\bar{Z}$

$B = X\bar{Y} + Y\bar{Z}$



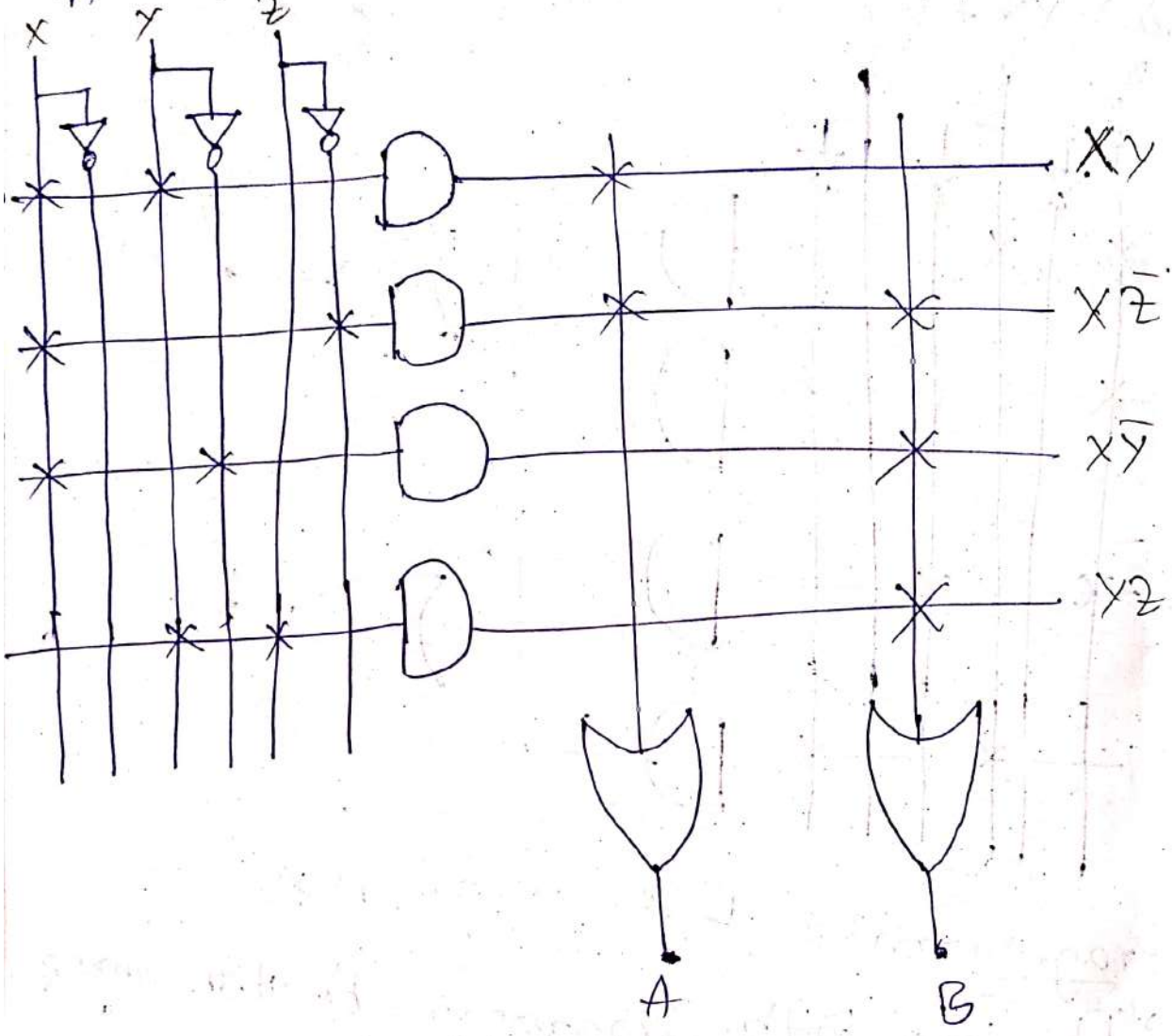
Programmable Logic Array (PLA)

→ It has both Programmable AND array & Programmable OR array.



Implementation Using Programming Logic Array.

$A = X\bar{Y} + XZ$ $\quad B = X\bar{Y} + YZ + X\bar{Z}$

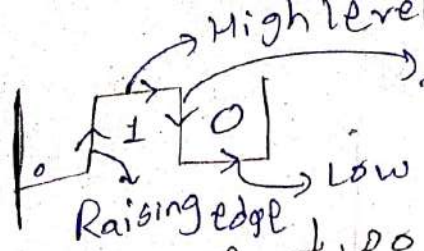


Five examples on PAL

1. $A = XY + Z$
2. $A = \bar{X}Y + \bar{Z}W$
3. $A = (X+Y)\cdot Z$
4. $A = \bar{X}Y + X\cdot Z$
5. $A = XY + \bar{X}Z$

Five examples on PLA

1. $A = XY + \bar{X}Z$
2. $A = (X+Y)\cdot(Z+W)$
3. $A = XY + \bar{X}\bar{Y}$
4. $A = (X+Y)\cdot(\bar{Z}+W)$
5. $A = \bar{X}Y + Z\cdot\bar{W}$

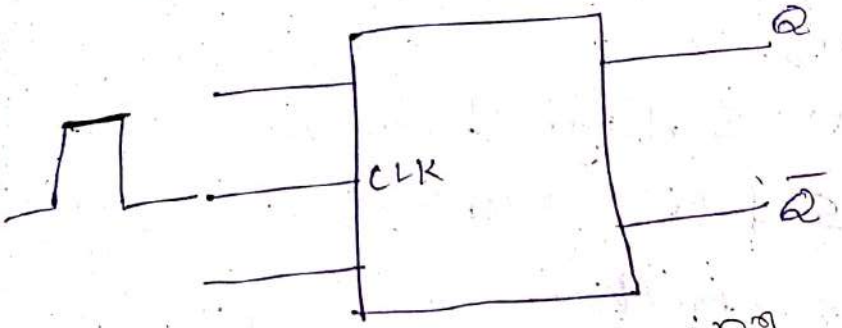


There are two types of Trigger signal.

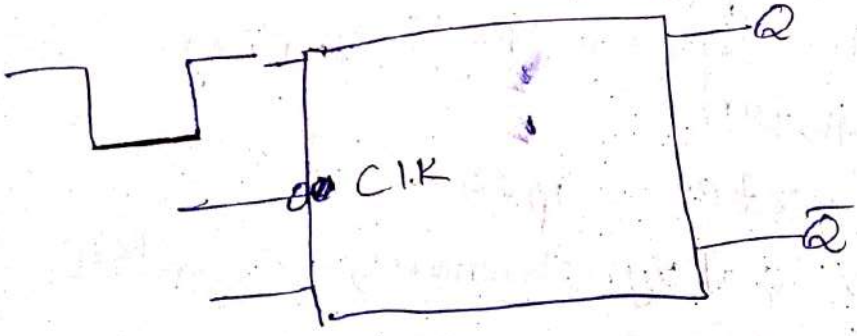
(1) Level Trigger signal
(2) Edge Trigger Signal.

(1) Level Trigger

(i) High Level Triggering

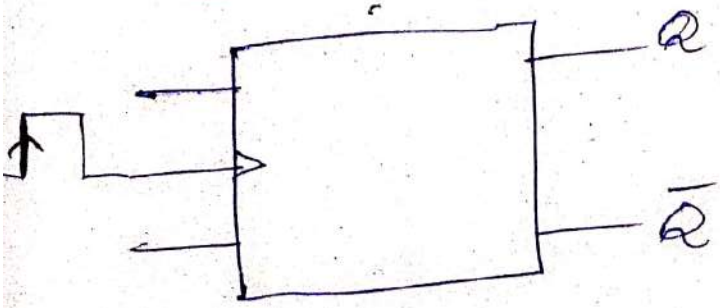


(ii) Low Level Triggering



(2) Edge Trigger

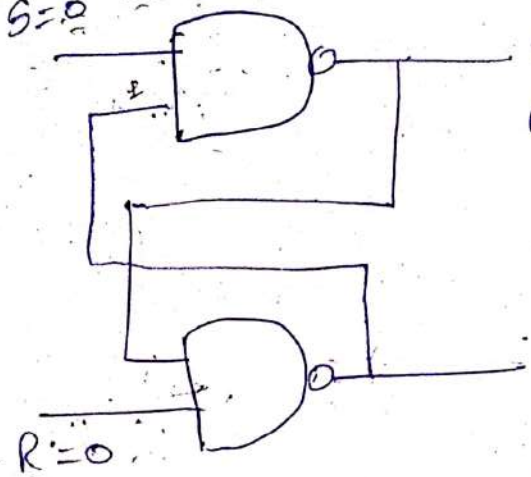
(i) Raising (or) Positive Edge



SR LATCH (NAND)

Case I:

S = 0



Q = 0 → 1 → 1

Q = 1 → 1 → 1

$\bar{Q}$ = 1 → 1 → 1

$\bar{Q}$ = 0 → 1 → 1

R = 0

S	R	Q
0	0	Invalid
0	1	
1	0	
1	1	

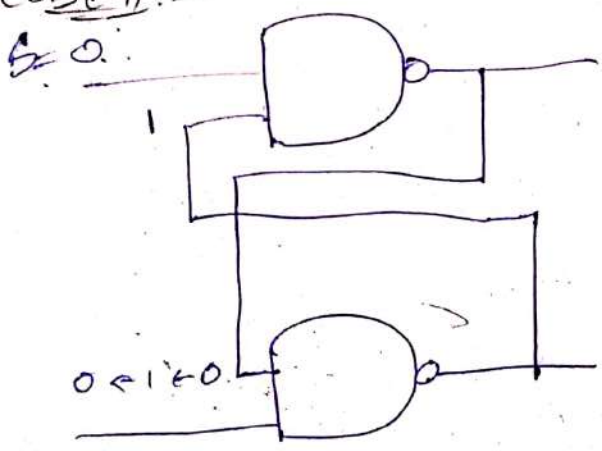
Latch

Note:- Stores temporarily

* No clock signal

Case II:-

S = 0



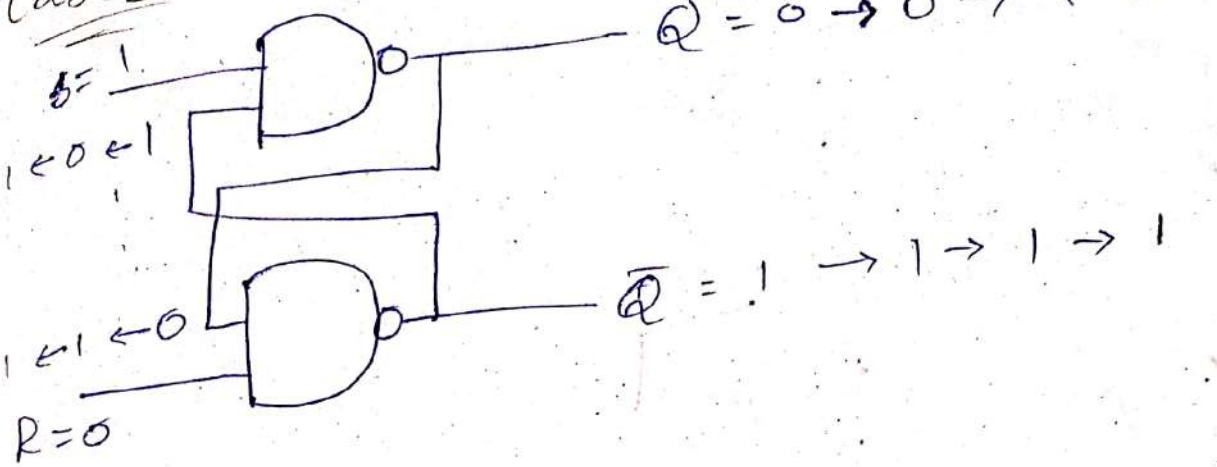
Q = 0 → 1 → 1 → 1

$\bar{Q}$ = 1 → 1 → 0 → 0

R = 1

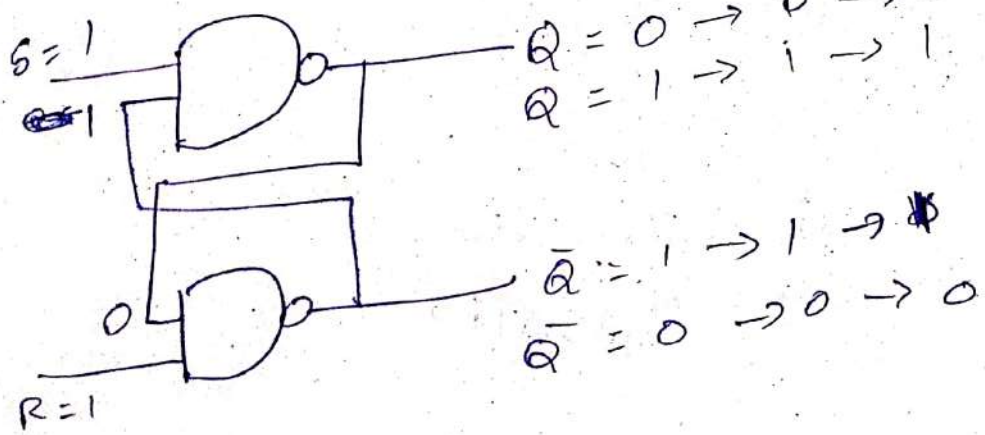
S	R	Q
0	0	Invalid
0	1	1
1	0	
1	1	

Case III :-



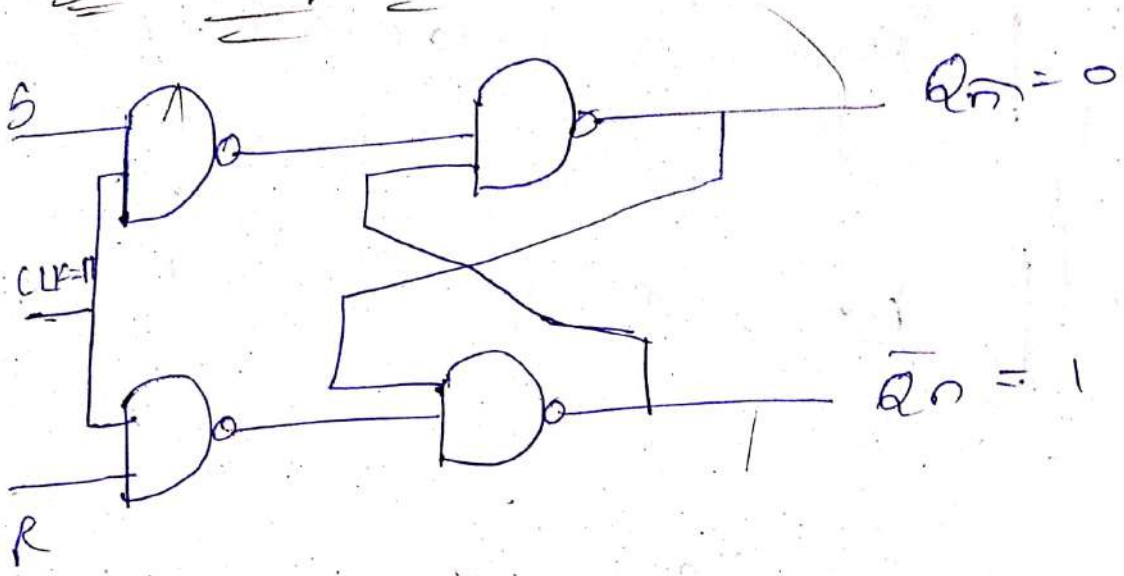
S	R	Q
0	0	Invalid
0	1	1
		0
1	0	
1	1	

Case IV :-



S	R	Q
0	0	Invalid
0	1	1
1	0	0
1	1	No change

SR Flip-Flop



S	R	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Invalid
1	1	1	Invalid

S	R	Qn+1	comment
0	0	Qn	No change
0	1	0	Reset
1	0	1	Set
1	1	X	Invalid

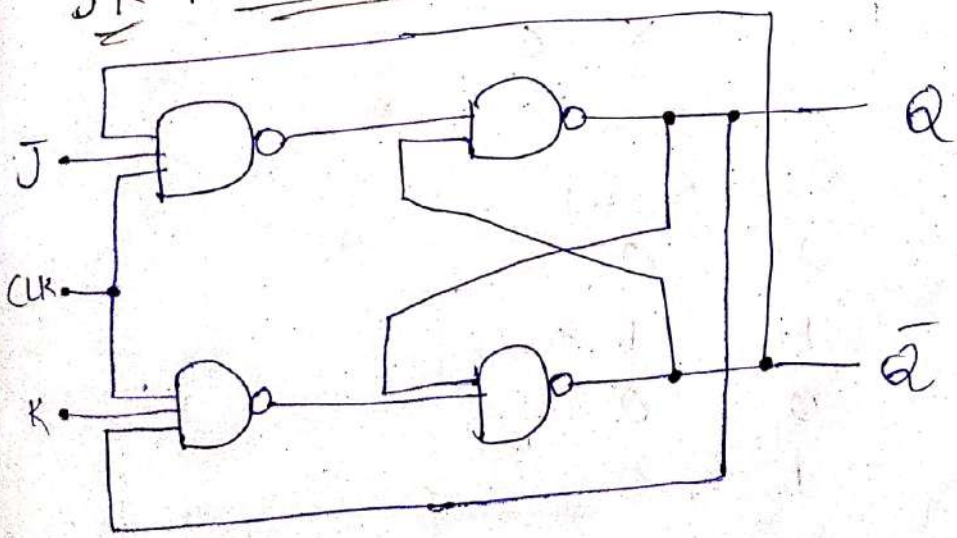
Characteristics Table

S	R	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	} for Invalid.
1	1	1	

Excitation Table

Qn	Qn+1	S	R
0	0	0	X (0,0,0.1)
0	1	1	0
1	0	0	1
1	1	X (00,10)	0

JK Flip-Flop



Characteristic table

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Excitation Table

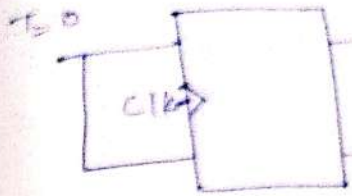
Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Characteristic Eq^n

$Q(t+1) = J\bar{Q}t + \bar{K}Q(t)$

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

T - flip flop (T



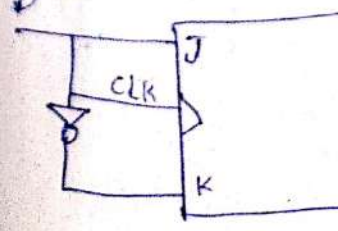
Characteristics

T	Q_n
0	0
0	1
1	0
1	1

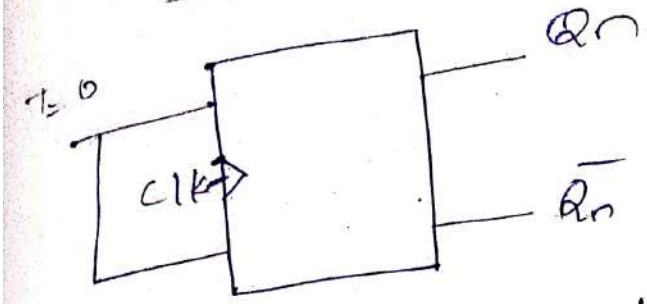
Excitation T

Q_n	Q_{n+}
0	0
0	1
1	0
1	1

Data Flip



T - flip flop (Toggle)



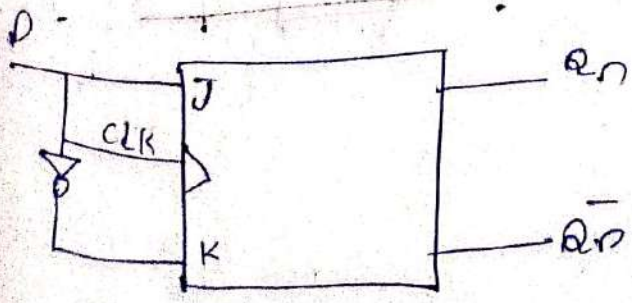
characterstics table

T	Qn	Qn+1	
0	0	0	} Qn
0	1	1	
1	0	1	} $\bar{Q}_n$
1	1	0	

Excitation Table

Qn	Qn+1	T
0	0	0
0	1	1
1	0	1
1	1	0

Data flip - flop (D-flip flop)



Characteristics table

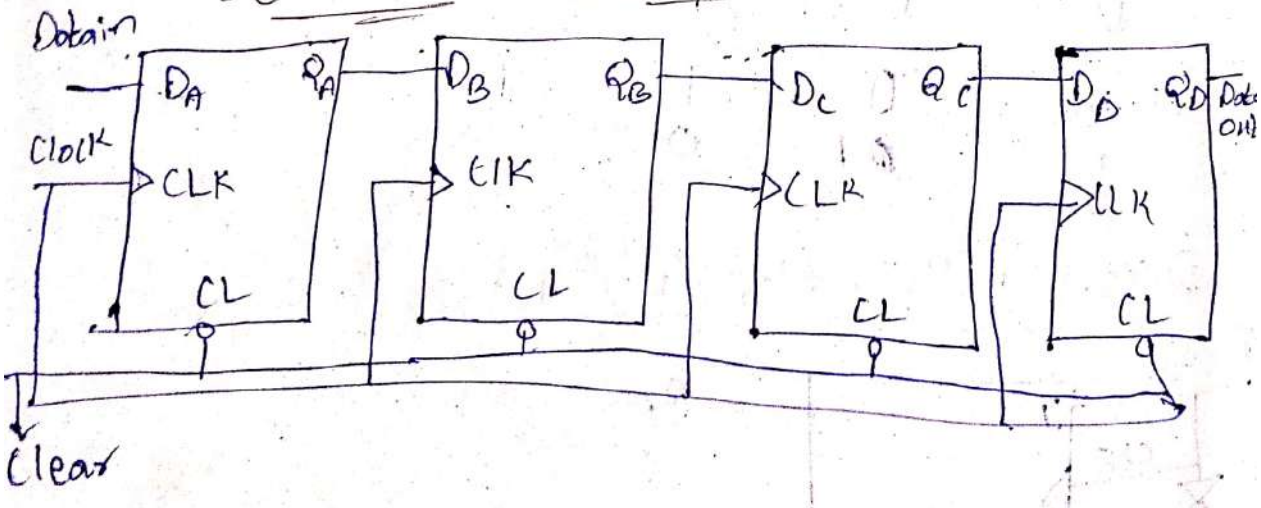
D	Qn	Qn+1
0	0	0
0	1	0
1	0	1
1	1	1

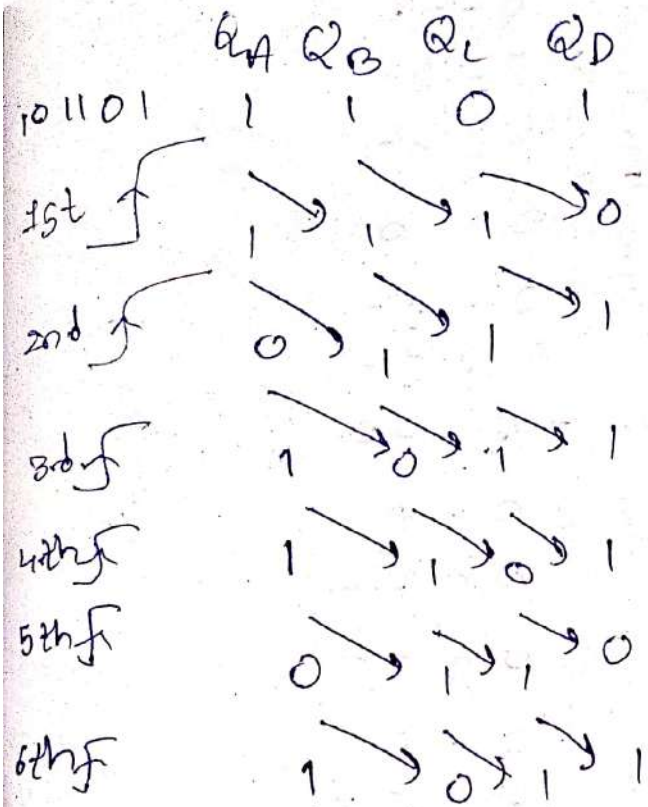
Excitation Table

Qn	Qn+1	D
0	0	0
0	1	1
1	0	0
1	1	1

Q. ~~Write down~~ A 4-bit shift register is initially filled with data 1101. The register is shifted six times to the right with the serial inputs.

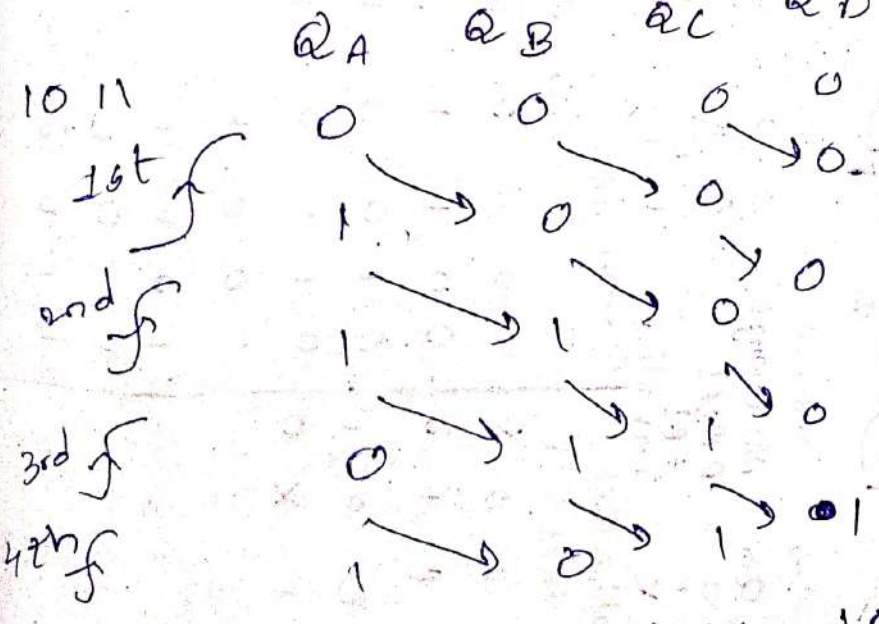
Serial-In & Serial-out



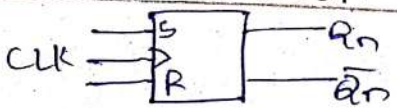
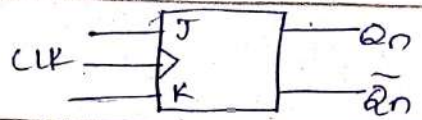
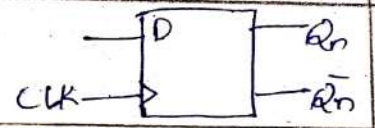
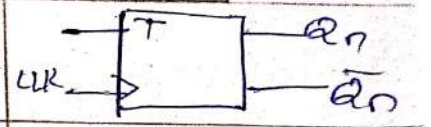
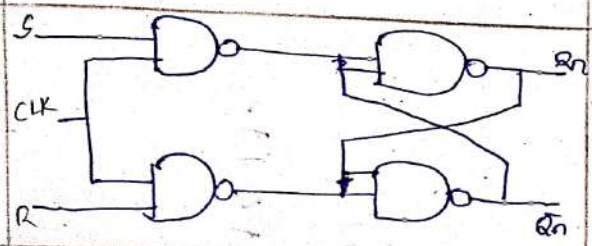
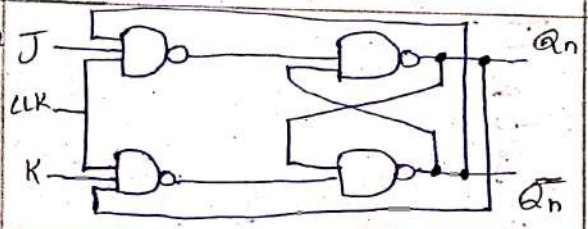
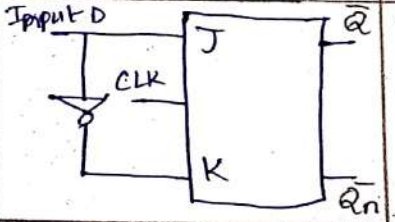
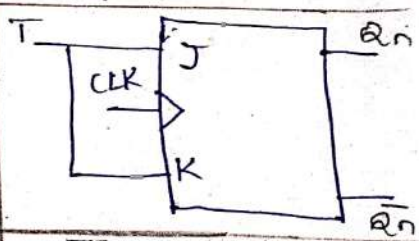


* After the 6th shift the output is 1011.

Serial - In & Parallel - out



* The final parallel out is 1011.

	SR Flip-Flop	JK Flip-Flop	D Flip-Flop	T-Flip flop
Logic symbol				
Circuit Diagram				
Truth Table	S R Qn+1 Comment 0 0 Qn No change 0 1 0 Reset 1 0 1 Set 1 1 X Invalid	J K Qn+1 Comment 0 0 Qn No change 0 1 0 Reset 1 0 1 Set 1 1 $\bar{Q}_n$ Complement	D Qn+1 0 0 1 1	T Qn+1 0 Qn 1 $\bar{Q}_n$

Characterstic (S-R)				Excitation (SR)			
S	R	Qn	Qn+1	Qn	Qn+1	S	R
0	0	0	0	0	0	0	X
0	0	1	1	0	1	1	0
0	1	0	0	1	0	0	1
0	1	1	0	1	1	X	0
1	0	0	1				
1	0	1	1				
1	1	0	Invalid				
1	1	1	Invalid				

Characterstic (JK)				Excitation (JK)			
J	K	Qn	Qn+1	Qn	Qn+1	J	K
0	0	0	0	0	0	0	X
0	0	1	1	0	1	1	X
0	1	0	0	1	0	X	1
0	1	1	0	1	1	X	0
1	0	0	1				
1	0	1	1				
1	1	0	1				
1	1	1	0				

Characterstic (D)			Excitation (D)		
D	Qn	Qn+1	Qn	Qn+1	D
0	0	0	0	0	0
0	1	0	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

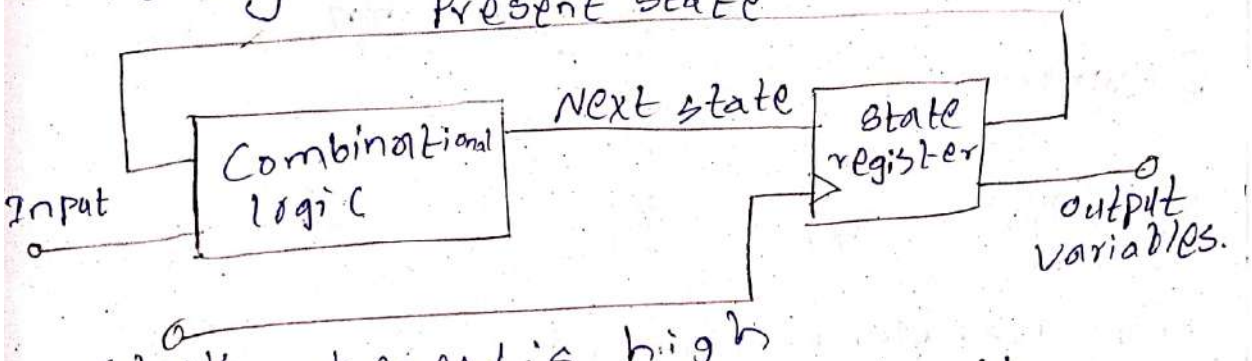
Characterstic (T)			Excitation (T)		
T	Qn	Qn+1	Qn	Qn+1	T
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	1	1	0

Types of sequential Circuits

① Synchronous sequential circuit.

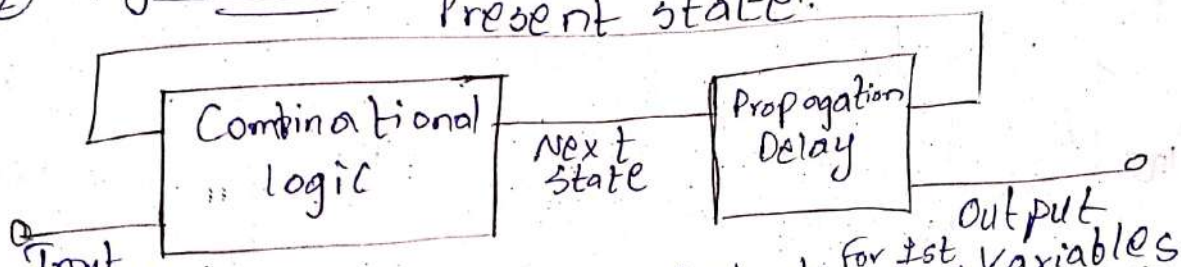
→ Change their state only at specific intervals, dictated by a clock signal.

→ Memory elements are clocked flip-flops.



*speed is high

② Asynchronous sequential circuit



→ ~~Do not rely on a clock signal~~ Clock signal is applied only for 1st Flipflop. for their operation, they change states immediately in response to changes in input.

→ Memory elements are unclocked flip-flops or time delay elements.

*speed is low.

Registers

→ To increase the storage capacity compared to flip-flop, we use group of flip-flops called as registers.

→ If we want to store an n-bit word, we have to use an n-bit register containing 'n' number of flip flops.

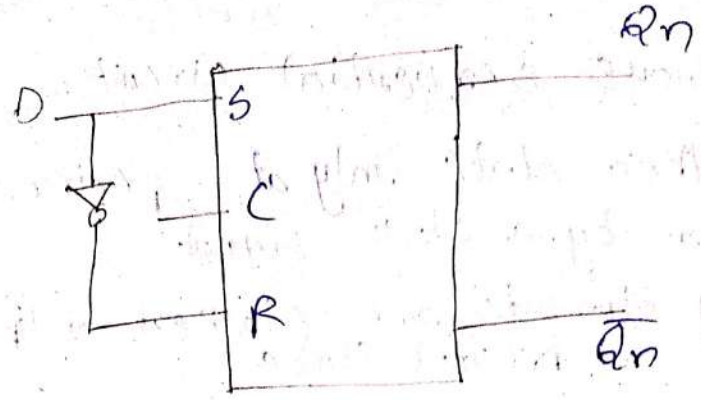
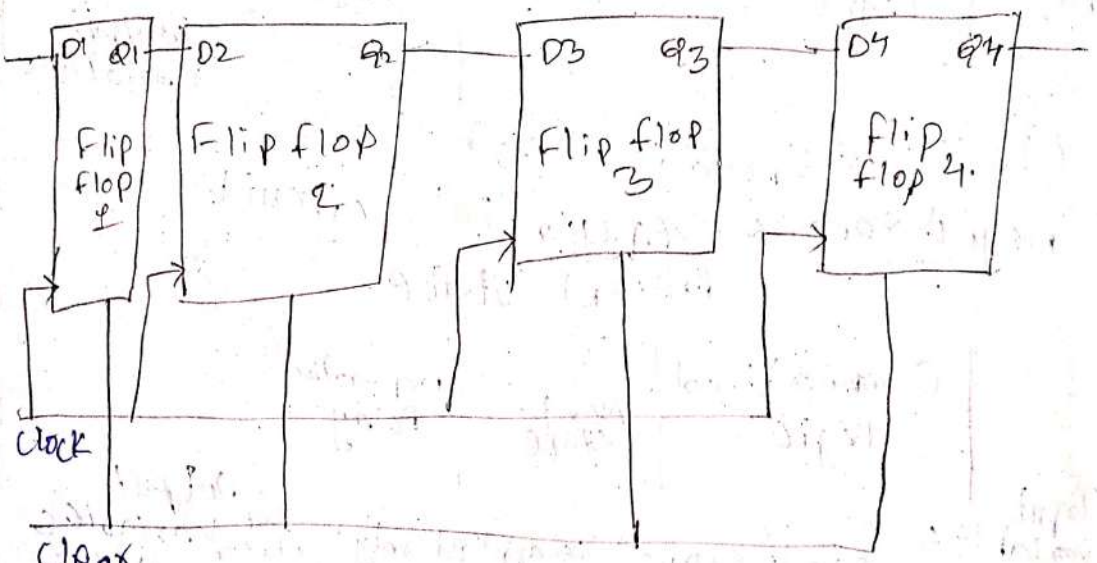


Diagram:-



Types of registers

→ Data registers:- Hold data for arithmetic and logic operations.

→ Address registers:- Store memory addresses

→ Status registers:- Indicates the status of the CPU: Contain flags.

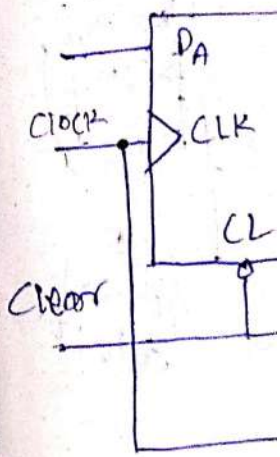
→ Instruction Register:- Holds the current instruction being executed.

Shift

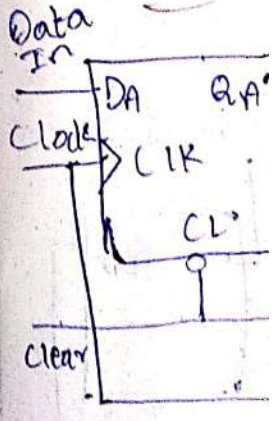
→ A Shi
data
flip-f

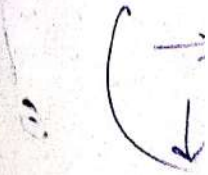
→ A shift
once e

Serial



Serial



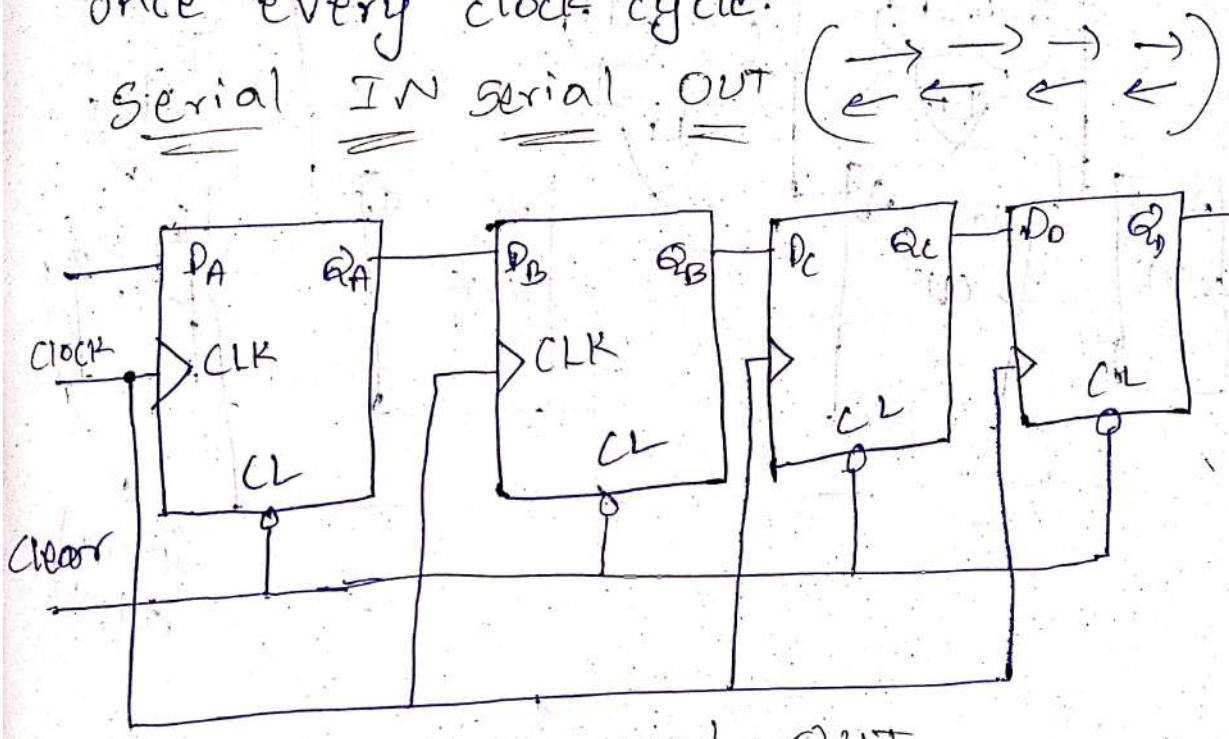


Shift register

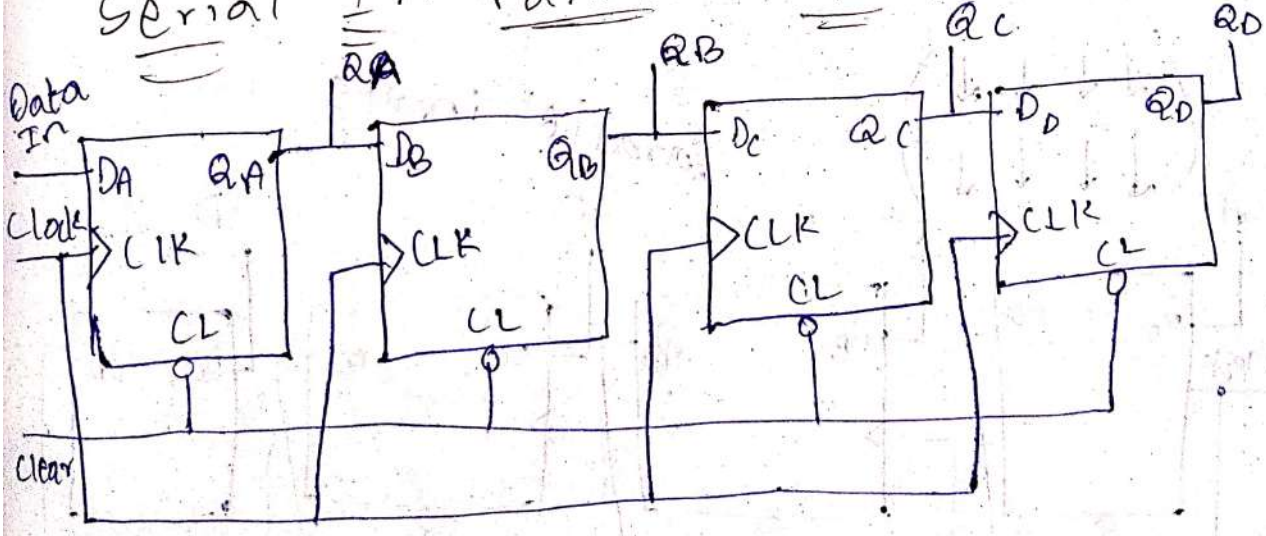
→ A shift register provides the data shifting function from one flip-flop to another.

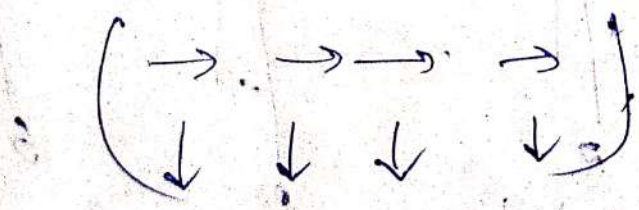
→ A shift register "shifts" its output once every clock cycle.

- Serial IN Serial OUT

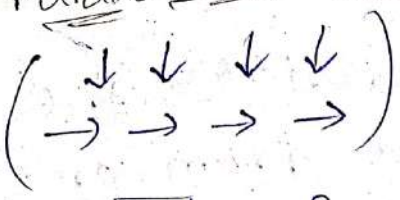


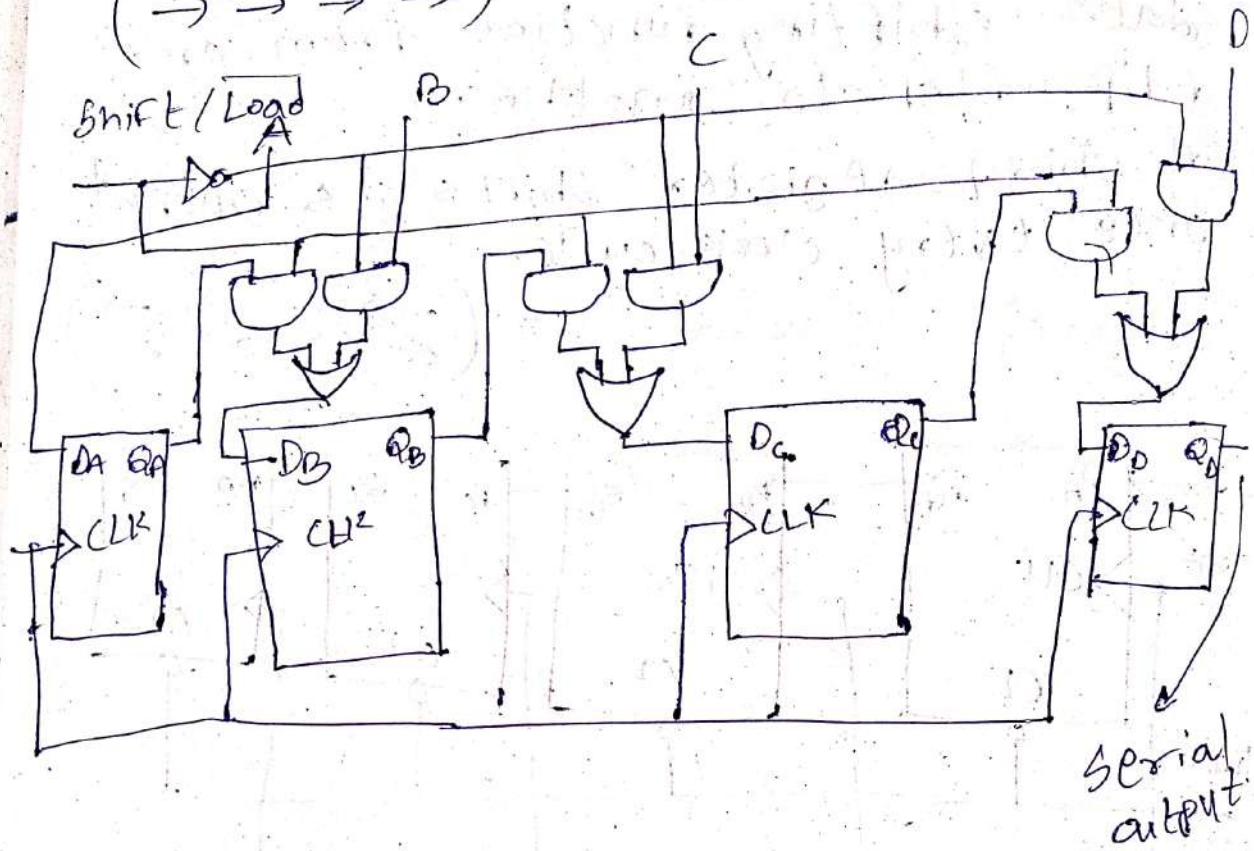
- Serial IN Parallel OUT



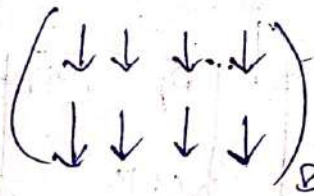


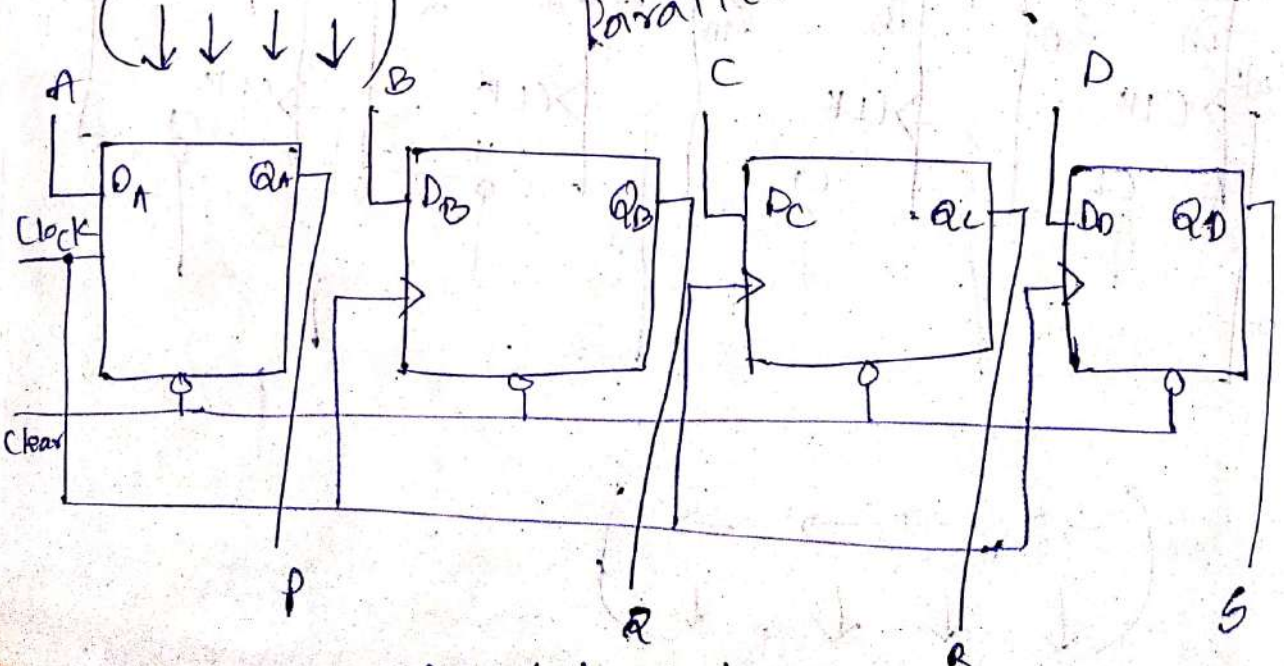
Parallel IN Serial OUT





Parallel IN Parallel OUT





Parallel outputs

Terminal Questions

① List the different types of shift registers.

A) * Serial-in, serial-out.
* serial-in, parallel-out.
* Parallel-in, serial-out.
* Parallel-in, Parallel-out.
* Universal.

② Illustrate the purpose of a clear and reset pin on a shift register.

A) → 'Clear' pin usually sets all the register outputs to 0.
→ 'Reset' pin may return the register to an initial state or configuration, depending on the specific shift register.

③ A 4-bit register is initially filled with the data "1101". The register is shifted six times to the right with the serial input being 101101. What is the content of the register after each shift?

A)

1 0 1 1 0 1

	Q_A	Q_B	Q_C	Q_D
	1	1	0	1
1st rs	1	1	1	0
2nd rs	0	1	1	1
3rd rs	1	0	1	1
4th rs	1	1	0	1
5th rs	0	1	1	0
6th rs	1	0	1	1

④ Construct a 4-bit shift register config. that outputs the binary data pattern "1011" in parallel format after receiving it serially.

A) 1011

	Q_A	Q_B	Q_C	Q_D
first	0	0	0	0
	1 →	0 →	0 →	0
f2nd	1 →	1 →	0 →	0
f3rd	0 →	1 →	1 →	0
f4th	1 →	0 →	1 →	1

*The final output is 1011 i.e; the input itself.

⑤ Design a 4-bit parallel-to-serial data converter using a PISO shift register.

A) for example the given input data is 0101

0 1 0 1 → 1
0 → 0 → 1 → 0 → 0
0 → 0 → 0 → 1 → 1
0 → 0 → 0 → 0 → 0

(or)

1000

1 0 0 0 → 0
0 → 1 → 0 → 0 → 0
0 → 0 → 1 → 0 → 0
0 → 0 → 0 → 1 → 1

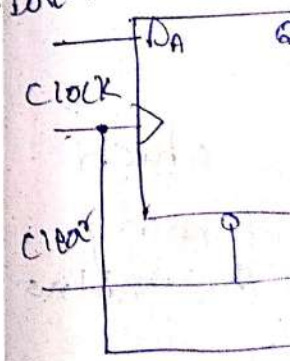
⑥ Construct
detects
in a se
starts w

A) 1 0 1 0

1st f
2nd f
3rd f
4th f

*After
out shi

Data In



⑦ Describe
operatio
shift r

A) The di
regist

I → →
→ →

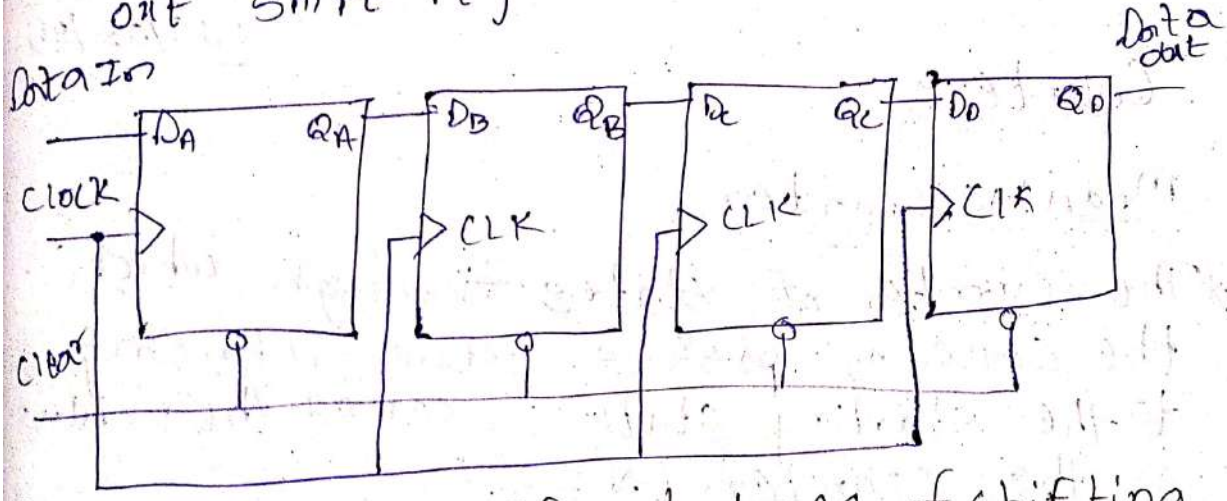
II ← ←
← ←

⑥ Construct a 4- bit shift register that detects a specific bit pattern of "1010" in a serial input stream. Initially it starts with "0110".

A) 1 0 1 0

	Q_A	Q_B	Q_C	Q_D
	0	1	1	0
1st f	0	0	1	1
2nd f	1	0	0	1
3rd f	0	1	0	0
4th f	1	0	1	0

*After 4 shifts in a serial-in & serial-out shift register.



⑦ Describe the different types of shifting operations that can be performed using shift registers.

A) The different operations using shift registers are as follows:-

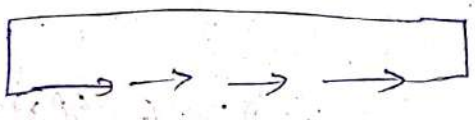
① → → → → } serial-in & serial-out
→ → → → } (right shift).

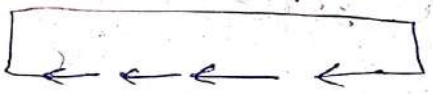
② ← ← ← ← } serial-in & serial-out
← ← ← ← } (left shift).

③ → → → → } Serial-in & Parallel-out
↓ ↓ ↓ ↓

④ ↓ ↓ ↓ ↓ } Parallel-in & Serial-out
→ → → →

⑤ ↓ ↓ ↓ ↓ } Parallel-in & Parallel-out
↓ ↓ ↓ ↓

⑥  ⇒ Rotate right.

⑦  ⇒ Rotate left.

19/02/25

Counters

Modulus Counter

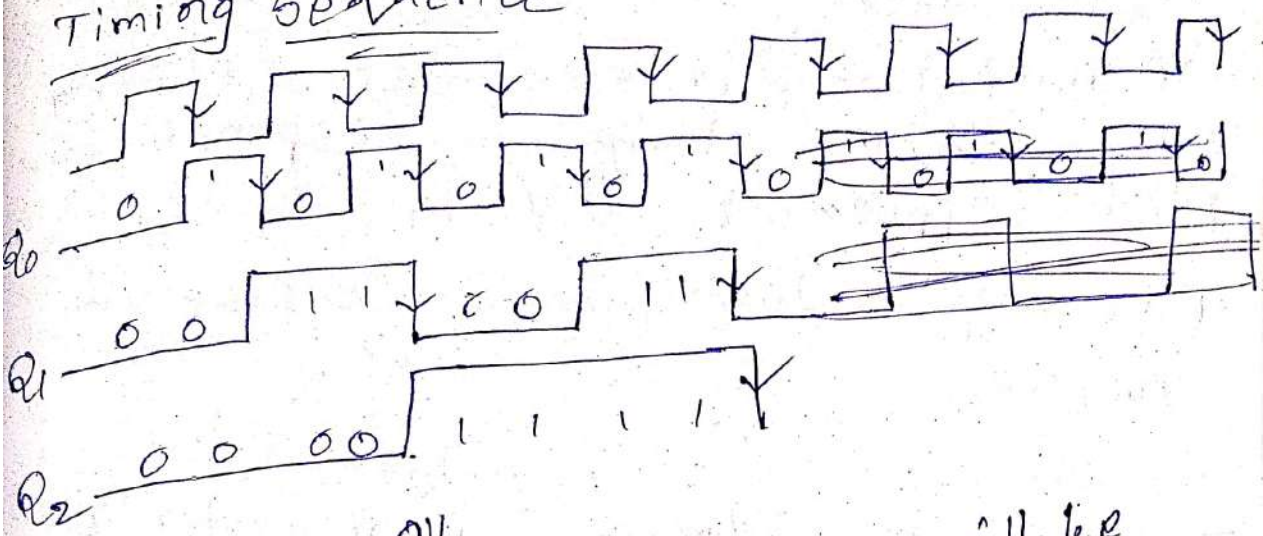
* The number of states through which the counter passes before returning to the starting state is called the modulus of the counter. ($N \leq 2^n$)

* Counters use JK Flip flop

Outputs of Mod 8 Ripple

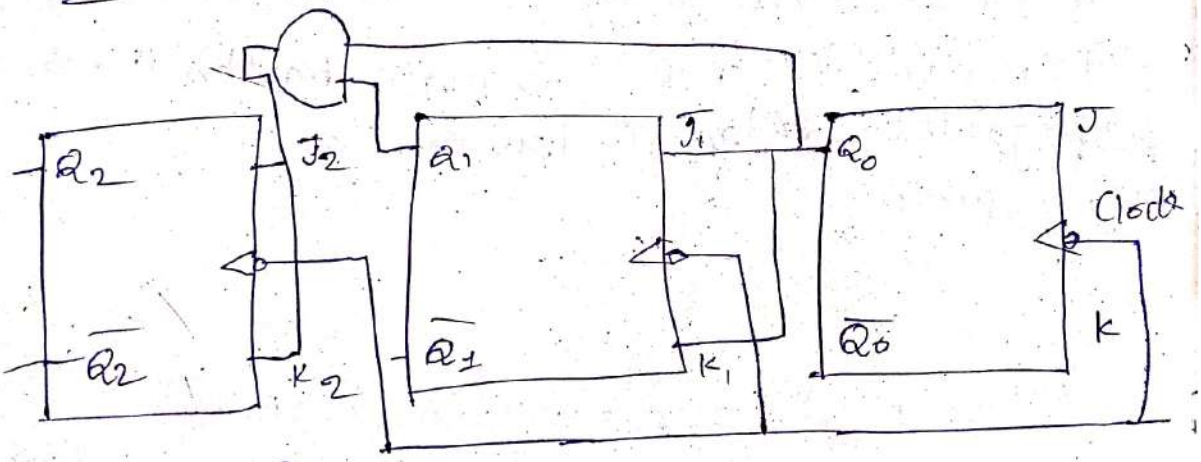
Q_2	Q_1	Q_0
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Timing Sequence



* Clear is '0' all the flipflops will be cleared and set to zero.

Mod 8 counter (3-bit synchronous)



Q_2	Q_1	Q_0
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

* Synchronous counters are known as parallel counters.

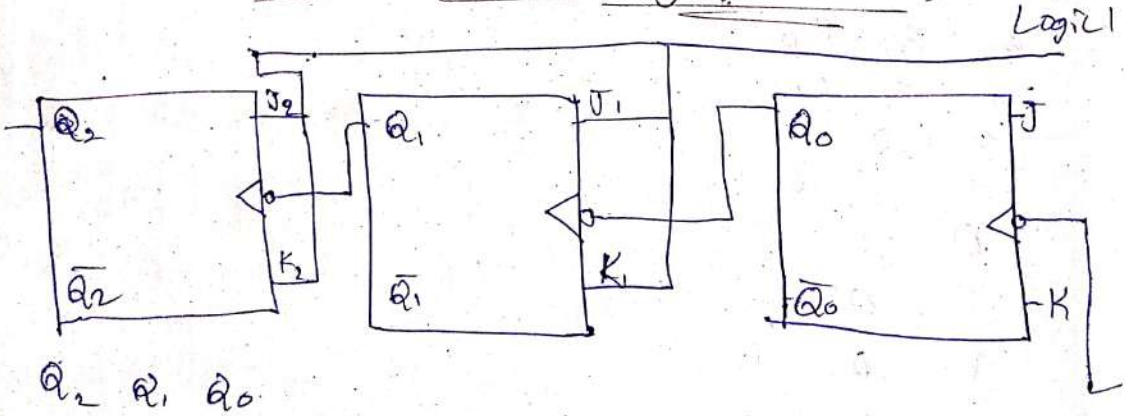
Digital Counter

→ A digital counter is a set of flip-flops whose states change in response to pulses applied at the input.

Difference between Synchronous & Asynchronous Counters

Synchronous	Asynchronous.
→ No connection between output of first flip-flop and clock input of next flip-flop.	output of first flip-flop drives the clock for the next flip-flop.
→ All the flip-flops are clocked simultaneously	→ All the flip-flops are not clocked simultaneously.
→ There will be no propagation delay i.e; high speed	→ main drawback is low speed.

Mod 8 Ripple Counter (Asynchronous)



Q_2	Q_1	Q_0
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

BCD R

*In this
using
clear f

Ex:- 0 –

Q_3
0
0
0
0

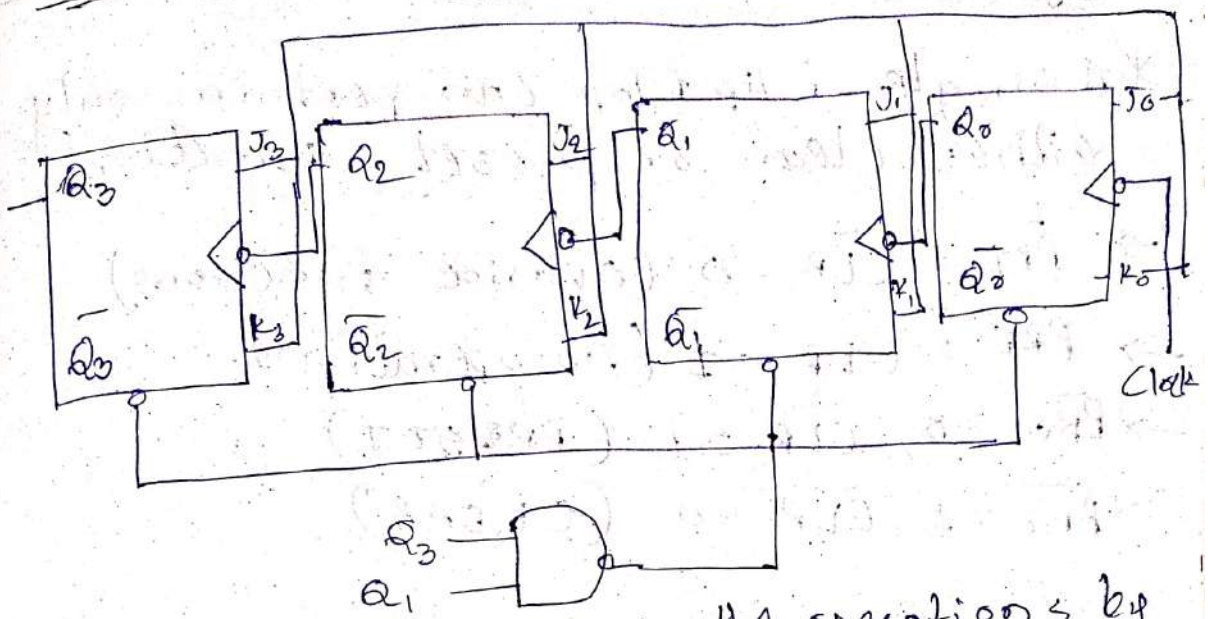
Advanta

→ Synchr

→ With a
is n

→ Overall
compa

BCD Ripple counter



*In this we can limit the operations by using the NAND gate i.e; which performs clear functions.

Ex:- 0 - 9

Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1

Advantages of synchronous counters.

→ Synchronous counters are easier to design.
→ With all clock inputs wired together there is no inherent propagation delay.
→ Overall faster operation may be achieved compared to Asynchronous counters.

Ring Counter

*A single flip flop can perform only either clear or preset function.

→ $\overline{PRE} = \overline{CLR} = 0$ (override functions)

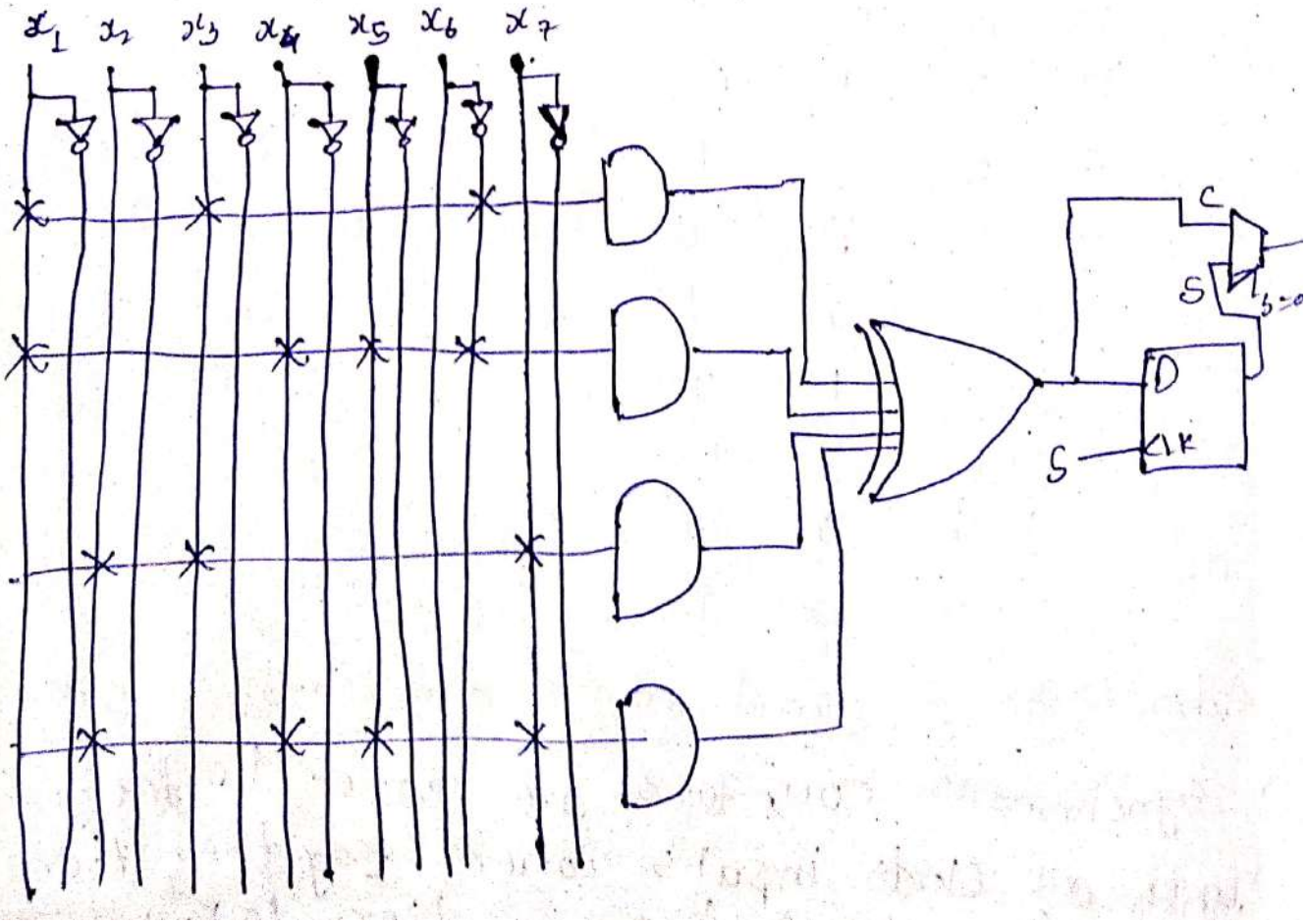
→ $\overline{PRE} = \overline{CLR} = 1$ (No operation).

→ $\overline{PRE} = 0, \overline{CLR} = 1$ (PRESET)

→ $\overline{PRE} = 1, \overline{CLR} = 0$ (CLEAR)

CPLD (Complex programmable Logic Device)

Q.) Implement the given Boolean function using CPLD $F = x_1 x_3 \bar{x_6} + x_1 x_4 x_5 \bar{x_6} + x_2 x_3 x_7 + x_2 x_4 x_5 x_7$

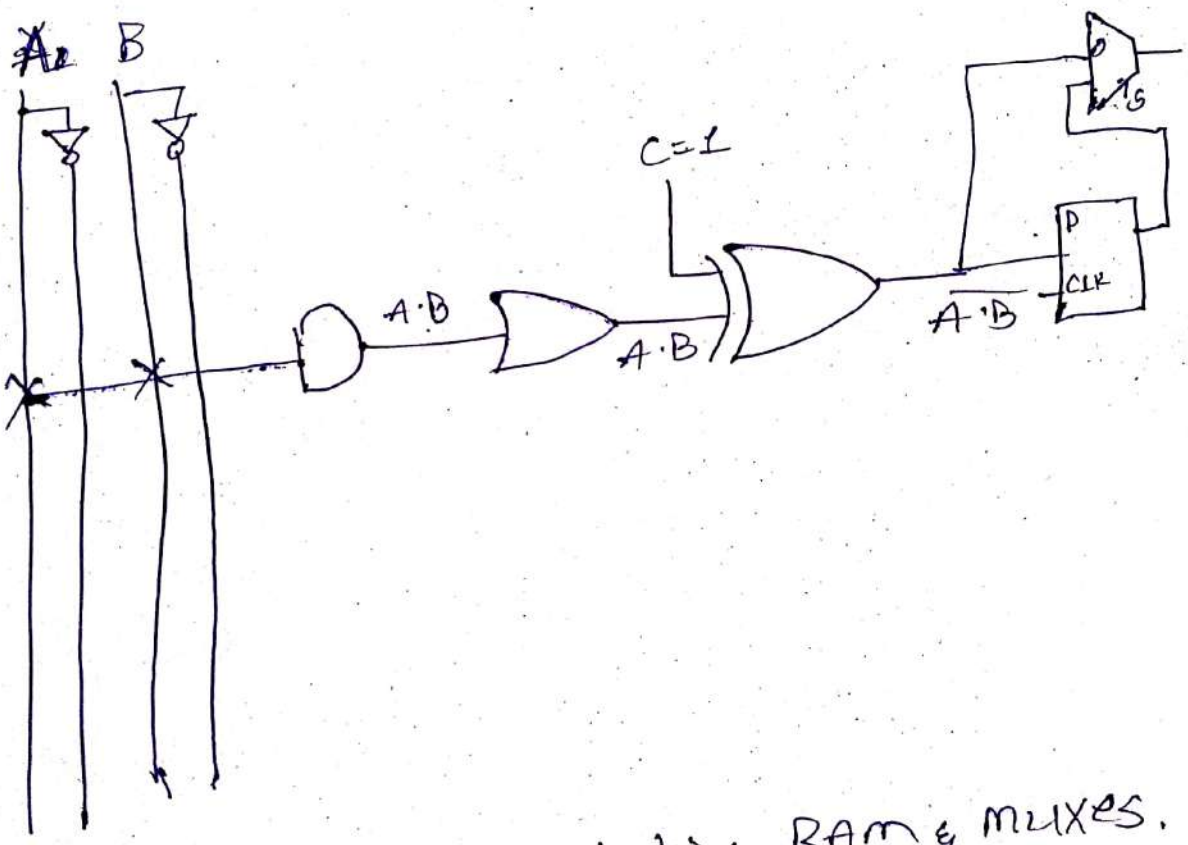


a) Implement the Boolean function $Y = \overline{A \cdot B} = \Sigma m(0,1,2)$ using macrocell (CPLD)

$0 = \bar{x}_1 \bar{x}_2 \bar{x}_3$

$1 = \bar{x}_1 \bar{x}_2 x_3$

$2 = \bar{x}_1 x_2 \bar{x}_3$



* CLB's contain static RAM & MUXES.
  ↓
  Configurable Logic Blocks.
* In CPLD we use PAL but in FPGA we use CLB's.

CO-3

Examples of Data Instruction format

1. MOV R1,R2
2. LOAD, 100(R4)
3. STORE, 200(R6)
4. IN R7, PORT 1
5. OUT PORT 2, R8

Examples of arithmetic instruction format

1. ADD R_1, R_2
2. ADD AX, [50H]
3. SUB BX, [50H + 40H]
4. MUL AX, BX
5. DIV AX, [BX]

Examples of Logical instruction format

1. AND R1, R2, R3 $\Rightarrow$ $R_1 = R_2 \& R_3$
2. OR R1, R2, R3 $\Rightarrow$ $R_1 = R_2 | R_3$
3. XOR $R_1, R_2, R_3 \Rightarrow R_1 = R_2 \wedge R_3$
4. NOT $R_1, R_2 \Rightarrow R_1 = \sim R_2$
5. NEG $R_1, R_2 \Rightarrow R_1 = -R_2$

Examples of Input & Output instruction formats

1. IN AX, 1234 H
2. OUT 50H, AX
3. MOV R1, [1234H]

Machine Cycle

15/03/2

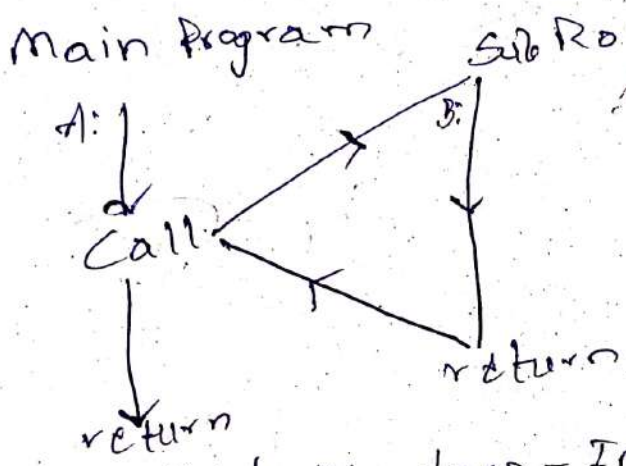
* For executing one instruction, 1 mach cycle is executed.
* One machine cycle has 4 clock-pulses:

1st clock-pulse ⇒ Fetch
2nd clock-pulse ⇒ Decode
3rd clock-pulse ⇒ Execute
4th clock-pulse ⇒ store.

* For an instruction there could be more than one machine cycle.
* PC → Program Counter
* Address location of the instructions is stored. Especially stores the address of next instruction.
* MAR = Memory Address Register.
* PC instructs MAR to identify the next instruction address location then MAR finds the address location.
* MBR = Memory Buffer Register.
* The identified address location is placed into Instruction Register (IR) by MBR.
* The above process is for fetch.

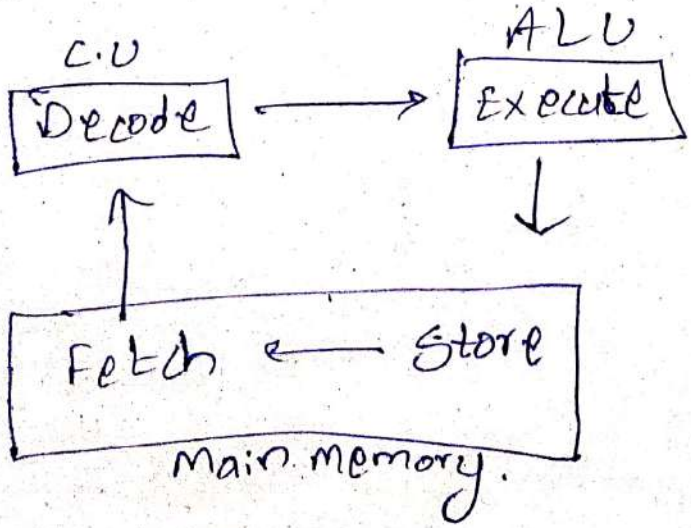
Sub-Routine (or) Sub program

* When we call another operation or program while the execution of main program then it terminates the main program and moves to the given operation or program after the completion of given program it returns back to the main program.



** This is called as loop-Inside-loop (or) nested loops.

* Fetch & Store are in ⇒ Main Memory.
* Decode ⇒ control unit
* Execute ⇒ ALU



19/03/25

RISC & CISC

* RISC => (Reduced Instruction Set Computing)
* CISC => (Complex Instruction set Computing)

→ In RISC Hardware usage is less where as instruction usage is more.

→ In RISC instructions are executed at high speed with most instructions completing in one cycle

CO-4

02/04/25

→ There won't be any data in the cache memory initially.

→ Registers > Cache > Main Memory > Secondary Memory

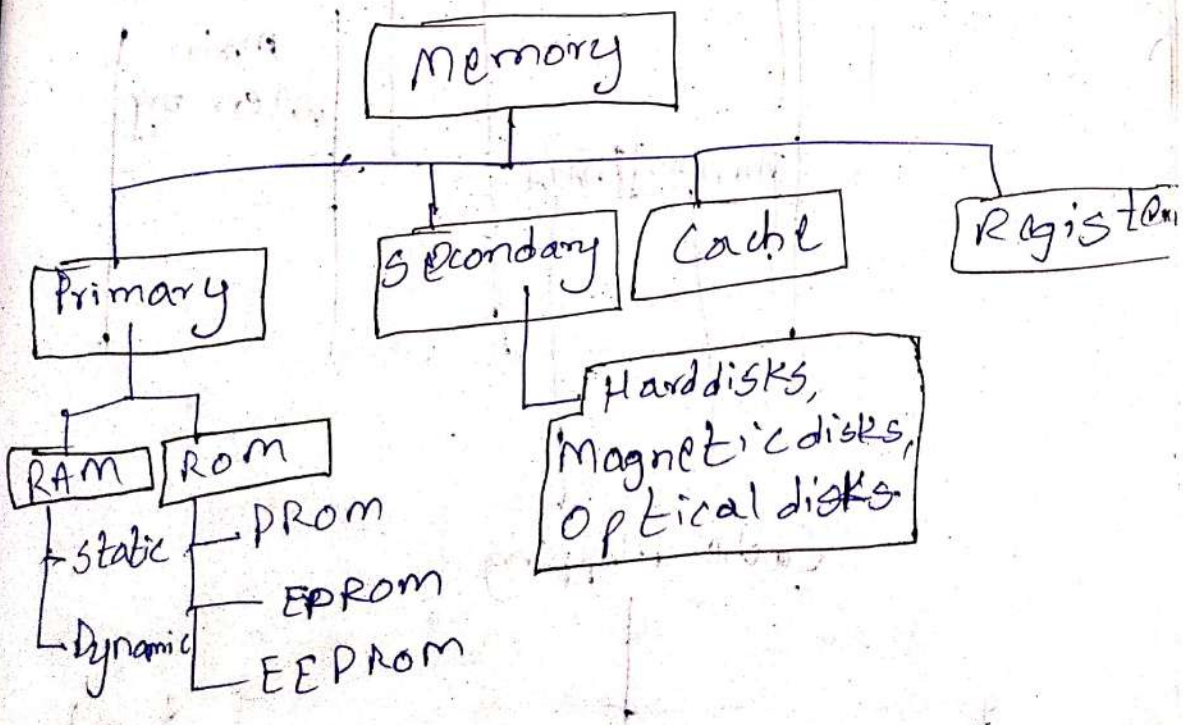
⇓

* This is in terms of speed. where the data is accessed from diff types of memory.

→ Registers < Cache < main memory < Secondary Memory

⇓

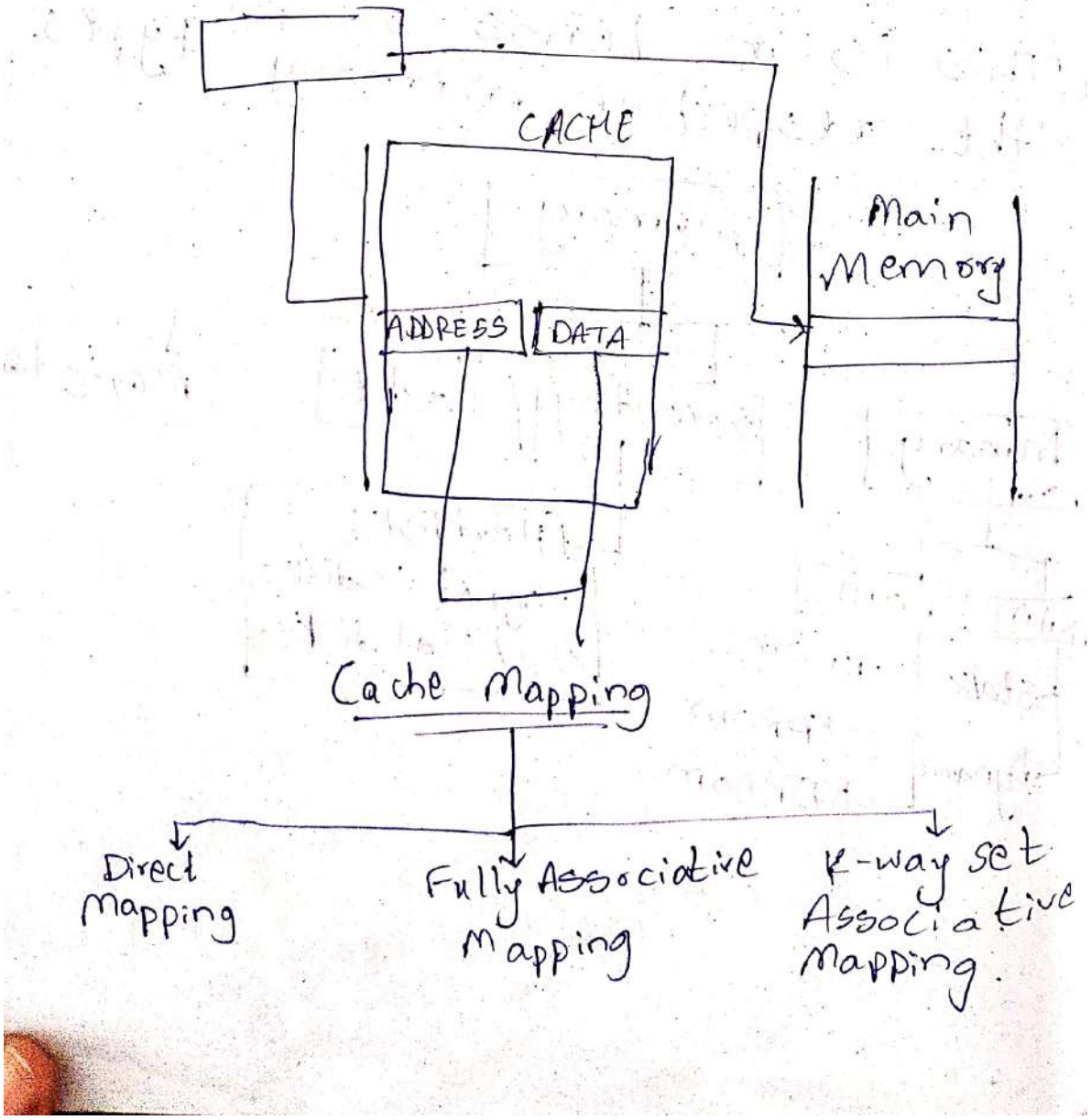
* This is in terms of size of the respective memory types.



*Cache memory is a faster, smaller section of memory with an access time that is comparable to registers.

Benefits of Cache

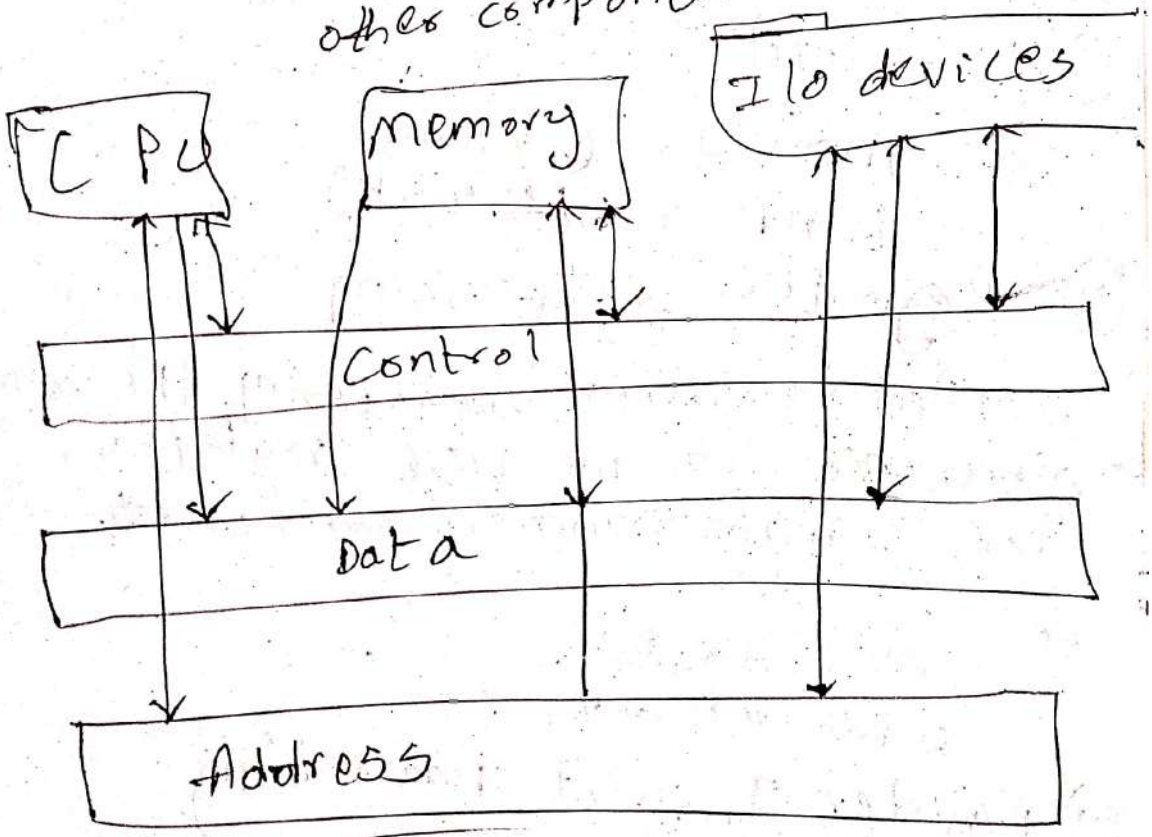
→ Faster access
→ Reducing memory latency.
→ Lowering bus traffic
→ Increasing effective CPU utilization.
→ Enhancing system scalability.



Address Bus :-
Carries memory addresses from the processor to other components such as primary storage & i/o devices.

Data Bus
Carries data b/w processor & other components.

Control Bus :- Carries control signals from the processor to other components.



Zero-Address | OP-code

One-Address | OP-code | Address-1

Two-Address | OP-code | A-1 | A-2

Three-Address | OP-code | A-1 | A-2 | A-3

① Immediate Addressing Mode

Ex:- mov AX, 2000H

Here source operand is always data.

② Direct

Ex:- mov AX, [1592H]

mov BL, [0300H]

→ Address field contains the effective address of the operand.

③ In-Direct

Ex:- mov AX, @2005H

LOAD R1, (2345H)

④ Register Addressing

Similar to direct addressing the only difference is we use registers than a main memory address ps.

Ex:- mov AX, BX.

ADD R1, R2

⑤ Register Indirect Addressing

Ex:- ADD AL, [BX]

mov AX, [BX]

⑥ Displacement Addressing

Combination of direct & register indirect addressing.

Instruction set Types

1. Data Transfer
2. Arithmetic
3. Logical
4. Input Output
5. System Control.
6. Transfer control.

1. Data transfer

- MOV => mov AX, BX
- STR (store) => STR T
- Load => LOAD A, [R1]
- Exchange => xchg ah, al
- Clear (reset) => clr ax
- Set => set A.

2. Arithmetic

Add => add al, 07h
Sub => sub ah, 05h
MUL => mul ax, 1234h
DIV => div ax, 8003h

3. Logical

AND => AND AL, 01h
OR => OR AH, 05h.
NOT => ~~AX~~.
XOR

④ I/O instruction set

* Input (Read) → I/O port to destination
* Output (Write) ⇒ Source → I/O Port
* Start ⇒ instructs I/O processor to initiate I/O operation
* Test I/O ⇒ status information from I/O system

⑤ System Control

→ These are used for system setting
⇒ These are used for Operating systems.

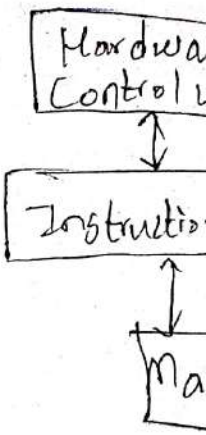
⑥ Transfer of control

① Branch
② Skip
③ Procedure call.

BRP X ⇒ Branch to location X → positive
BRN X ⇒ " " → Negative
BRZ X ⇒ " " → Zero
BRO X ⇒ " " → overflow

RISC (Reduced Instruction set Computing)

* Simple instructions are executed.
* objective is to execute instructions at a high speed.
* Through pipelining instructions are executed faster.
* operations are performed between registers

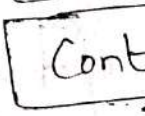


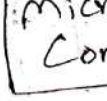
Disadvantages

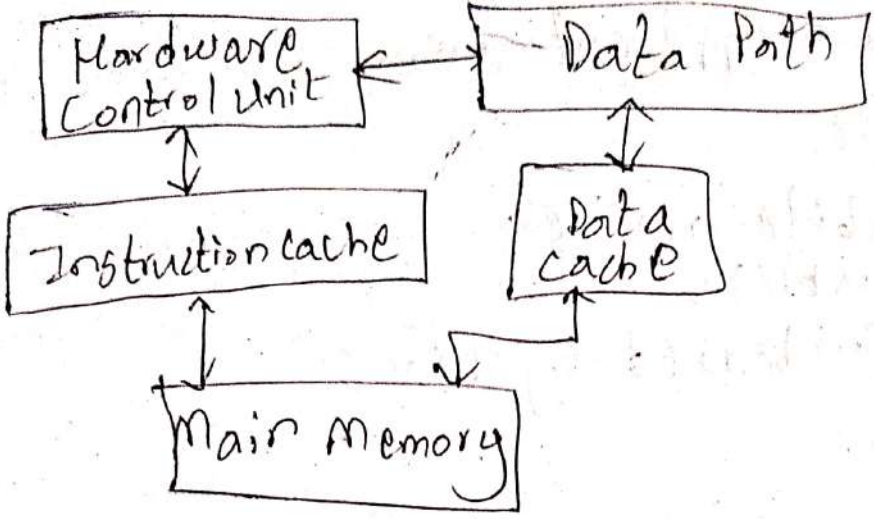
→ Requires mor operations.
→ Compiler has instruction

CISC (Comple

* Large set o
* By fewer are capable
* Reduces
* No need of







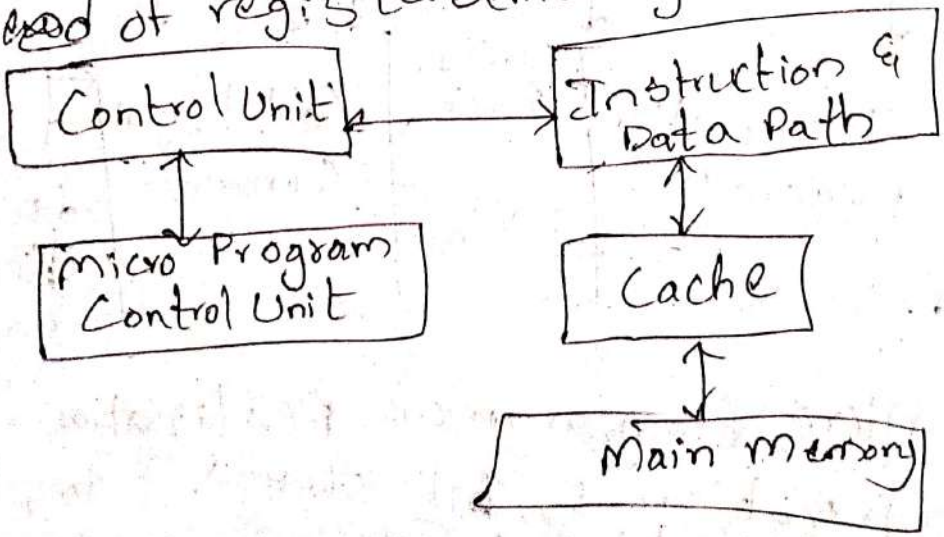
Disadvantages

→ Requires more instructions for complex operations.

→ Compiler has to work harder to optimize instruction usage.

CISC (Complex Instruction Set Computing).

* Large set of complex instructions.
* By fewer lines of code, instructions are capable of multiple operations.
* Reduces the software complexity..
* No need of registers (memory-to-memory).

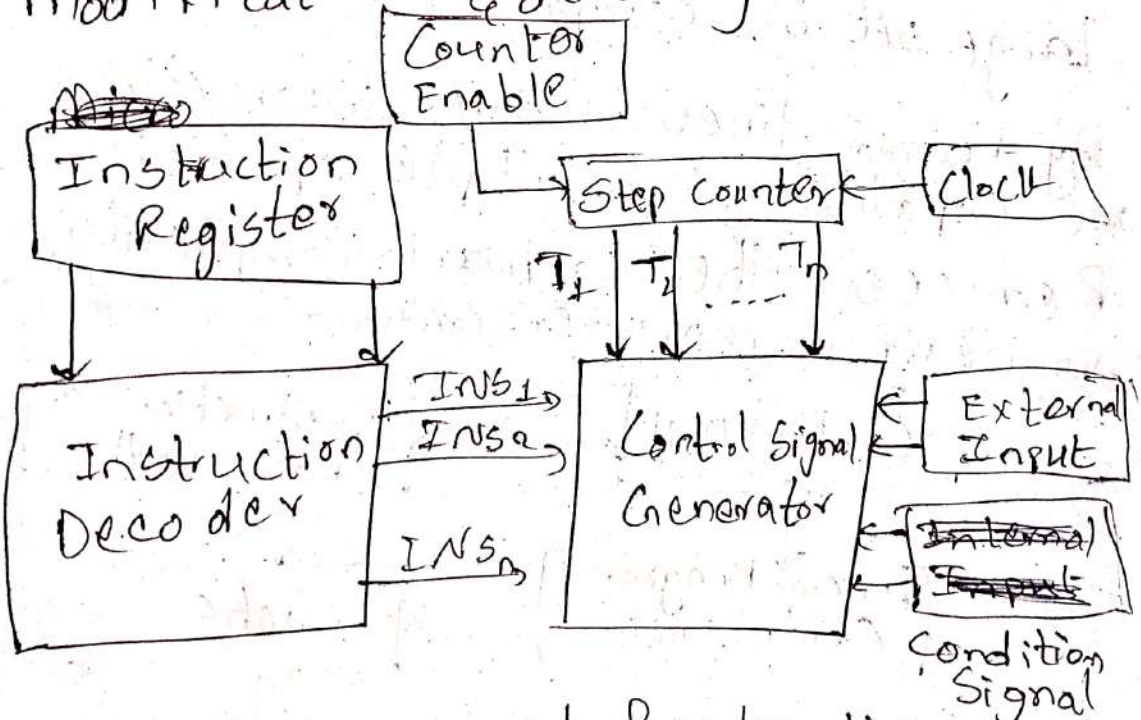


Applications

RISC
→ Mobile devices
→ Tablets
→ Embedded systems

CISC
→ Server processors
→ Intel's x86 architecture.
→ Ryzen.

Hardwired Realization
→ Execution is faster, implementation, modification & decoding are difficult



Micro Programmed Realization
→ Execution is bit slower, implementation modification & decoding are easier.
→ The n-bit words are contained by each micro instruction. On the basis of the bit pattern of a control word

every control signals differ from each other.

→ TO execute a particular instruction in microprogrammed realization, we need a sequence of micro instructions. This is called micro routine.

Multicycle Implementation

→ Divison of a long cycle of the single cycle implementation into 3 to 5 shorter cycles.

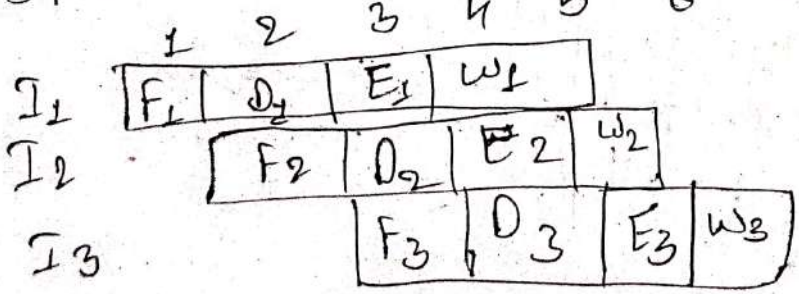
* No of cycles depend on instructions.

Characterstics

→ Reuse of Hardware Components.
→ Variable Execution time.
→ Control logic complexity.
→ Efficiency & Performance.

Pipelining

→ Pipelining is a technique of decomposing a sequential process into sub-operations, with each sub-process being executed in a special dedicated segment.



FI DI CO FO EI WO

FI DI

Pipeline Hazards

→ If there are several issues and challenges in the execution of pipelining then it is called a pipeline hazard.

Types

① Structural Hazards

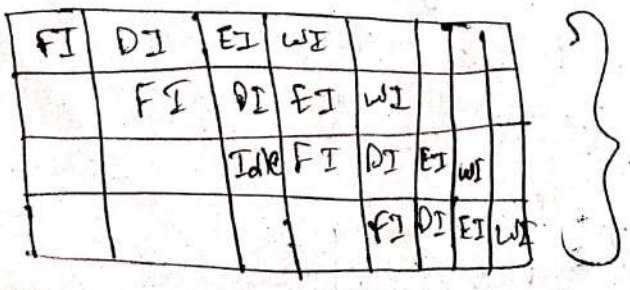
When the conflict occur between the sources of hardware components then there would be conflict in executing an instruction simultaneously.

② Data Hazards

They occur when an instruction depends on the result of previous instructions.

③ Control Hazard

→ If the program counter changes due to pipelining of branches or other instructions.



Structural Hazards.

~~Data~~ CS

FI DI FO EI WO

FI DI Idle

FI

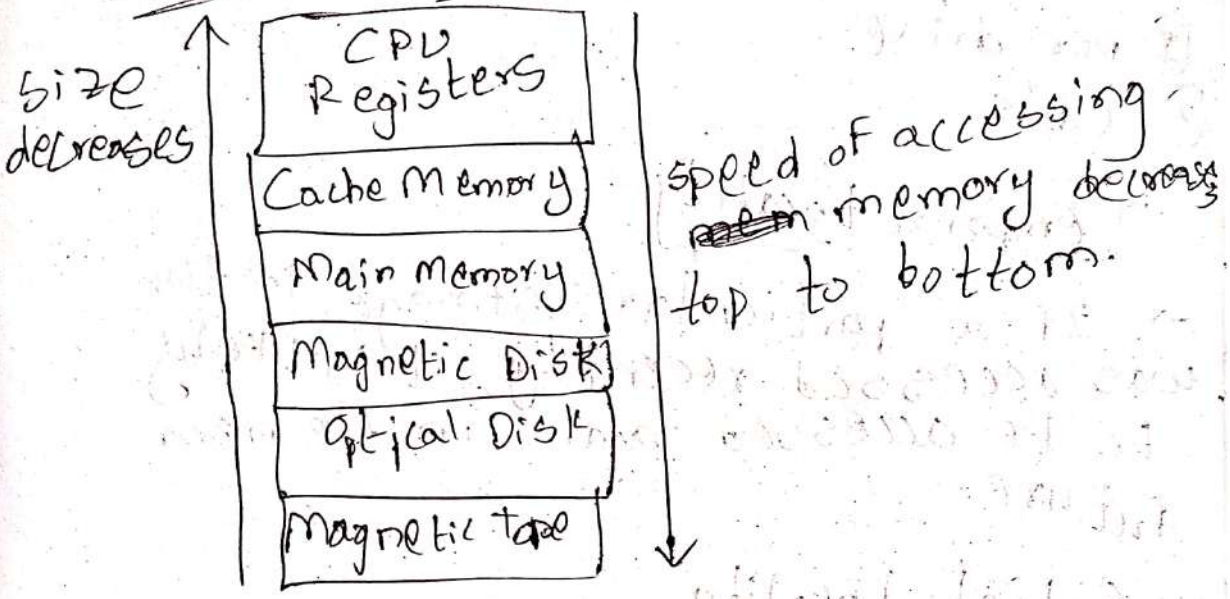
FO EI WO

DI FO EI WO

FI DI FO EI WO

CO-4

Storage Systems



Random Access Memory

→ Used to store information temporarily

→ Data will be lost once the computer turned off.

Static RAM

Retains stored information as long as the power supply is ON.

Dynamic RAM

Stores the binary data in the form of electrical charge applied to capacitors.

Fixed Storage

1. Hard Disk
2. SSD
3. Internal flash memory.

Removable Storage

1. Pen drive
2. CD's.

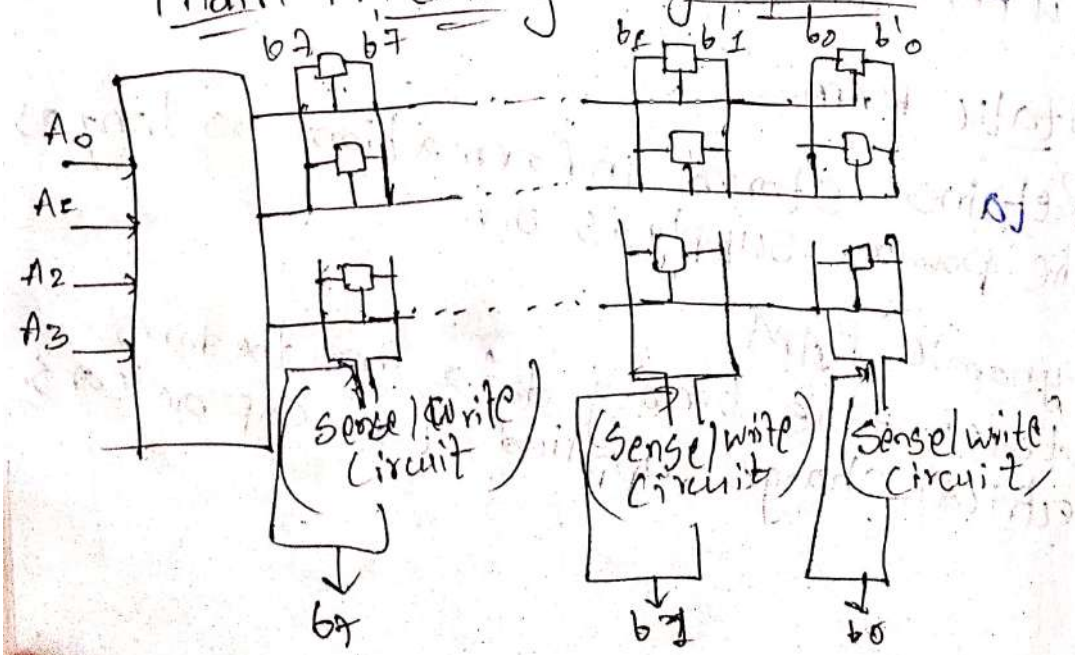
Temporal Locality

→ If a particular storage location was accessed recently, it is likely to be accessed again in the near future.

Spatial Locality

→ If a storage location is accessed, locations whose addresses are close by are likely to be accessed.

Main memory organization



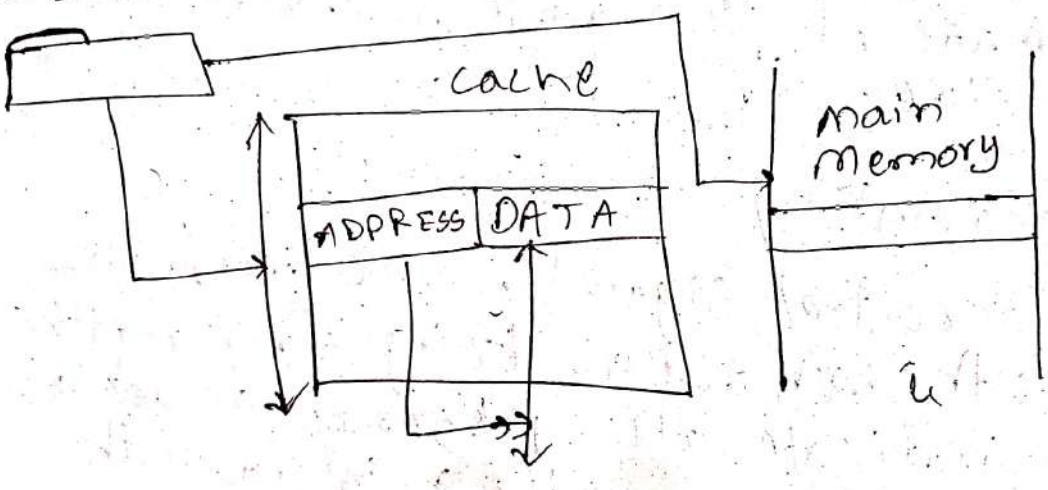
Cache Memory

→ It is a speedier, smaller section of memory with an access time that is less than comparable to registers.

* It serves as a buffer.

→ Improves overall system performance by reducing the average time taken to access data.

Working of a cache



→ Cache Mapping

* It is a technique using which the content present in the main memory is brought into the memory of cache.

Cache Mapping

- Direct Mapping
  → Directly accessed to cache without any replacement
- Fully Associative Mapping →
- k-way Set Associative mapping.

External Storage

→ HDD's

→ SSD's

→ USB flash Drives.

→ Magnetic Tape

Stores digital data which consists of a long, narrow strip of plastic coated with a magnetic material.

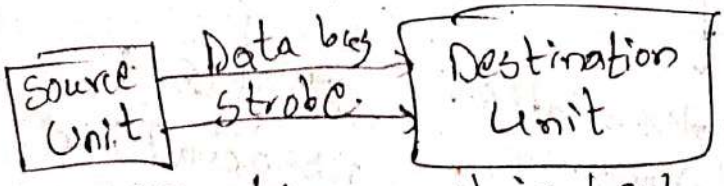
* Made of iron diode or Chromium dioxide.

Handshaking

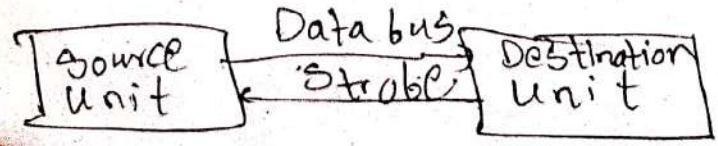
→ A control signal is accompanied with each data being transmitted to indicate the presence of data. The receiving unit responds with another control signal to acknowledge receipt of the data.

Strobe Control

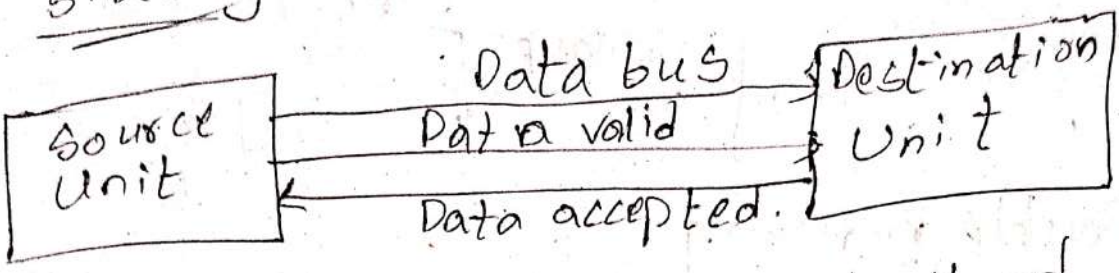
Source initiated Strobe



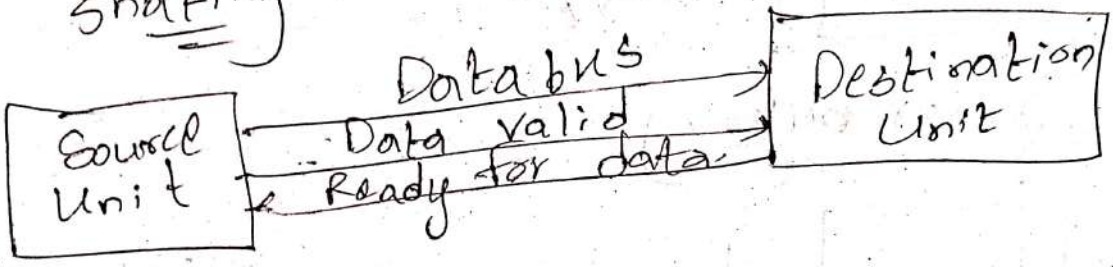
Destination initiated strobe



Source Initiated transfer using Hand-shaking



Destination-initiated using Hand shaking



* In O.S, buffering is a technique which is used to enhance the performance of I/O operations of the system.

→ Buffered data can be accessed be more quickly as compared to the original source of the data.

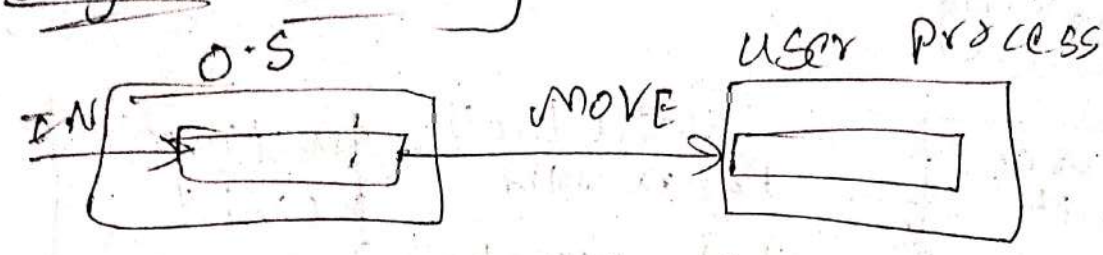
Major reasons for buffering in OS

→ It creates synchronization b/wn two devices having different processing speed.

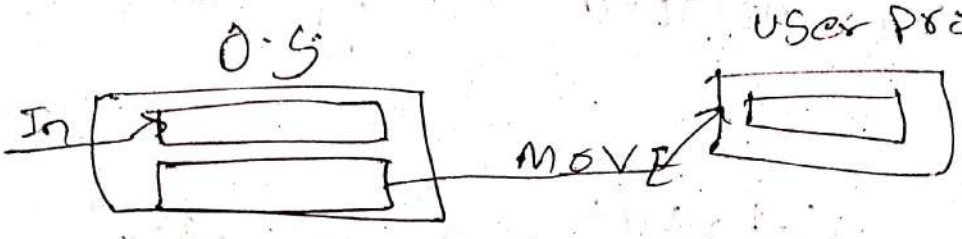
→ It is required when two devices have different data block sizes.

→ To support copy semantics for application I/O operations.

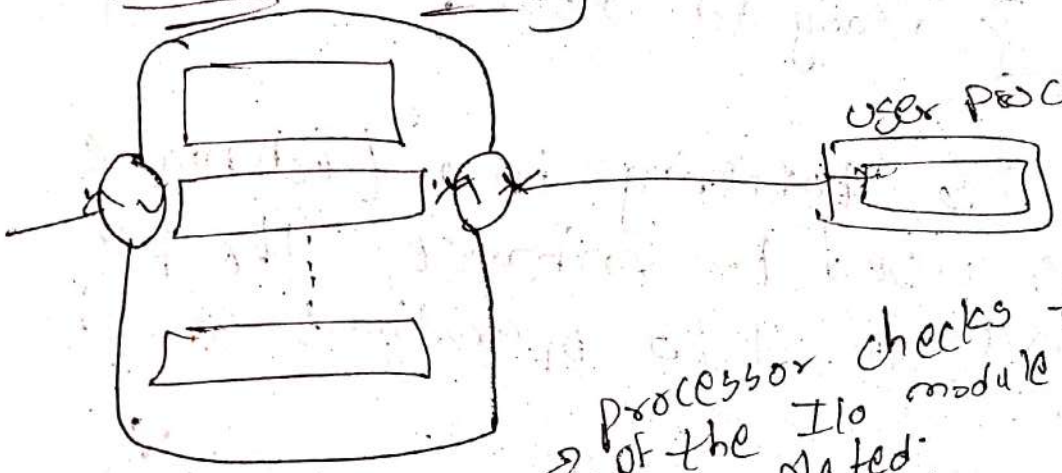
Single Buffering



Double Buffering



Circular Buffering



⇒ Programmed I/O technique → Processor checks the status of the I/O module until its completed.

I/O was the simplest I/O technique for the data exchang blwn the processor & external devices

⇒ Interrupt Drive I/O transfer is initiate by the interrupt command issued to the C.P.U.

* There is no need for the CPU to stay in the loop as the interrupt command interrupts the CPU when it was ready for data transfer.

Programmed I/O

→ Easy Implementation.
→ Requires less hardware support.

Advantages.

Disadvantages

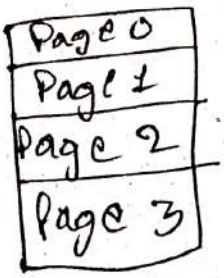
→ Ties up CPU for long period of time.
→ Busy waiting.

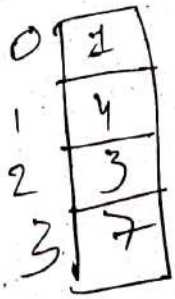
Interrupt I/O

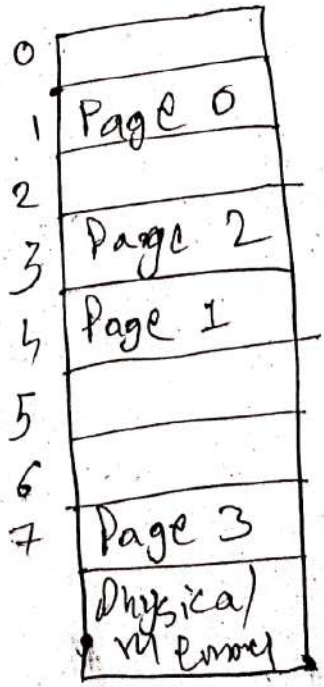
→ Fast
→ Efficient

Page Table:- The data structure that is used by the virtual memory system in O.S to store the mapping btwn the physical & logical memory.

EX:-







Translation Lookaside Buffer (TLB)

→ It is a special cache used to keep track of recently used page table entries that have been recently accessed.